



Escuela
Politécnica
Superior

Error Preservation in Automatic Speech Recognition for Language Learners: Benchmarking and Fine-Tuning on the Speak & Improve Corpus



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:

Felipe Rojas Cascante

Tutor/es:

Juan Antonio Pérez Ortiz

Mayo 2026



Universitat d'Alacant
Universidad de Alicante

Contents

1. Introduction	3
1.1. Context and Motivation	3
1.2. Objectives	3
1.3. Contributions	4
2. Theoretical Framework	6
2.1. Automatic Speech Recognition (ASR)	6
2.2. L2 Learner Speech	9
2.3. Computer-Assisted Language Learning (CALL)	10
2.4. ERRANT: Grammatical Error Taxonomy	10
3. Data and Resources	12
3.1. Speak & Improve Corpus	12
4. Benchmark Methodology	14
4.1. Loading the Speak & Improve Corpus	15
4.2. ASR Transcription	15
4.3. Normalization Before Scoring	16
4.4. Word Error Rate	17
4.5. Error Preservation Rate (EPR)	17
4.6. Grammatical Error Preservation (GEP)	18
4.6.1. Building the Targets	18
4.6.2. Classifying Each Edit	19
4.6.3. Three States, Not Two	20
4.7. A Worked End-to-End Example	20
4.8. Aggregation and Outputs	22
5. Evaluated Models	23
6. Results	25
6.1. Word Error Rate (WER)	25
6.2. Grammatical Error Preservation (GEP)	27
6.2.1. Analysis by ERRANT Error Type	28
6.2.2. Correction by Grammatical Category	33
6.3. WER–GEP Trade-off	34
6.4. Eval-Set Stability	35
6.5. Analysis by Speaker Proficiency Level	35
6.5.1. WER by Proficiency Level	37

6.5.2. GEP and Auto-Correction by Proficiency Level	38
6.5.3. WER–GEP Trade-off by Proficiency Level	39
7. Lightweight Models: Viability in Resource-Constrained Environments	40
7.1. Selected Models	40
7.2. Comparative Results	41
7.2.1. Parakeet TDT-CTC 110M: Exceptional Performance at Reduced Scale	42
7.2.2. Canary 180M Flash: Decoder Impact at Reduced Scale	42
7.2.3. Moonshine Base: Limits of Ultra-Lightweight Models	43
7.3. Efficiency–Performance Trade-off	44
7.4. Parakeet 110M vs. 1.1B: Comparison by Proficiency Level	45
7.5. Deployment Implications	46
8. Experimentation: Fine-Tuning for Error Preservation	48
8.1. Hypothesis and Design Rationale	48
8.2. Building the DPO Preference Pairs	50
8.3. Loss Function and Training Configuration	52
8.4. Training Behavior	54
8.5. Results on the Full Eval Partition	55
8.6. Where DPO Helps Most	56
8.7. Qualitative Examples	59
8.8. Why the Recipe Works	59
9. Discussion	61
9.1. Implications for Technology-Assisted Language Learning	61
9.2. Limitations	62
9.3. Future Work	62
10. Conclusions	64
11. References	66

1. Introduction

1.1. Context and Motivation

Modern Automatic Speech Recognition (ASR) systems have achieved remarkable accuracy levels for native speakers, with Word Error Rates (WER) below 5% in controlled conditions (Xiong et al., 2017). However, when these systems are applied to the speech of second-language (L2) learners, a fundamental problem arises that goes beyond simple transcription accuracy: the **silent auto-correction of grammatical errors**.

Language learners make systematic errors, including: incorrect use of prepositions, number agreement, wrong verb forms, and unnecessary articles, all of which are valuable diagnostic evidence for intelligent tutoring systems. When an ASR model transcribes “I arrive **to** the airport” (a preposition error: it should be *at*) as “I arrive **at** the airport”, the learner’s error may sometimes disappear from the transcription, depriving the system of critical information needed to generate feedback.

This phenomenon may be due to implicit language models embedded in the current state of the art models. This language model is capable of “correcting” the learner’s speech towards grammatically standard forms. Exploring different ASR architectures may help future work decide which models to choose and are less prone to this issue.

1.2. Objectives

This work aims to:

1. **Quantify** the magnitude of the auto-correction problem in contemporary ASR models, using real speech from English learners drawn from the Speak & Improve corpus (3,209 utterances, 10,397 grammatical errors in the eval partition).
2. **Compare** fundamentally different ASR architectures such as: Seq2Seq with attention (Whisper Small, Whisper Medium) and pure CTC (Wav2Vec2 Large, Parakeet CTC 1.1B) regarding their tendency to preserve or correct grammatical errors, and identify the current best model for language tutoring applications.
3. **Develop and validate an evaluation pipeline** integrating two complementary metrics: Word Error Rate (WER) and a novel Grammatical Error Preservation (GEP) metric based on the ERRANT toolkit.
4. **Analyze** which of the 39 ERRANT grammatical error types retained for GEP scoring are most susceptible to auto-correction, distinguishing between intentional corrections (where the model produces the grammatically standard form) and random mutations.

5. **Assess the influence of speaker proficiency level** on ASR performance by stratifying results according to continuous SLA scores (low, medium, and high bands), determining whether errors from lower-proficiency learners are disproportionately lost.
6. **Evaluate lightweight models** (Parakeet TDT-CTC 110M, Canary 180M Flash, Moonshine Base) for deployment in resource-constrained environments such as mobile devices, establishing whether the CTC architectural advantage over Seq2Seq holds at reduced parameter scales.
7. **Investigate the viability of fine-tuning** for improving error preservation through a targeted Direct Preference Optimization (DPO) recipe that constructs preference pairs from selectively curated utterances, demonstrating that focused-scope adaptation can improve GEP without degrading WER.

1.3. Contributions

The main contributions of this work are:

- A complete evaluation *pipeline* that integrates WER and a novel Grammatical Error Preservation (GEP) metric based on ERRANT, validated on 3,209 utterances and 10,397 scored grammatical errors (eval partition) with four ASR models.
- The identification of Parakeet CTC 1.1B (and its model family) as the optimal model for tutoring applications: it achieves the best WER (15.35%) **and** the highest grammatical preservation (77.7%), demonstrating that the CTC architecture combined with massive data outperforms Seq2Seq models across both axes.
- Evidence that autoregressive decoders are significant contributors to grammatical auto-correction but highly complex encoders such as Parakeet’s FastConformer has similar capabilities: Whisper Medium (16.4%) > Whisper Small (14.1%) > Parakeet CTC (12.8%) > Wav2Vec2 (2.9%).
- A granular analysis by ERRANT error type (39 scored types) identifying agreement errors (R:NOUN:NUM, R:VERB:SVA), morphology (R:MORPH), and verb forms (R:VERB:FORM) as the most vulnerable to auto-correction, with rates between 20 and 35%.
- An analysis by speaker proficiency level, based on continuous SLA scores, revealing that errors made by low-proficiency learners are preserved worse (up to 10.3 pp less than high-proficiency learners). This has direct implications for tutoring applications, where lower-level speakers are precisely those who need feedback the most.
- An analysis of lightweight models (61–182M parameters) demonstrating that Parakeet TDT-CTC 110M nearly reproduces the performance of the 1.1B model with only 10.7%

of its parameters (WER 15.67%, GEP 78.5%), validating its viability for deployment on mobile devices and in resource-constrained environments.

- A DPO *fine-tuning* recipe that improves Whisper Small on both axes simultaneously, lowering WER from 18.39% to 17.27% and raising GEP from 76.3% to 80.3% on the full eval partition. The recipe relies on selectively curated preference pairs (1,274 utterances filtered to only those where the baseline auto-corrected a whitelisted ERRANT edit) and an LLM-based minimum-edit restoration step that keeps both candidate transcriptions in Whisper’s native output style, so the loss never asks the model to change anything other than its preference at the error site.
- The full source code, pipeline scripts, evaluation results, and the Typst source of this document are publicly available at <https://github.com/FRoja1/TFG>.

2. Theoretical Framework

2.1. Automatic Speech Recognition (ASR)

Automatic Speech Recognition (ASR) is the process of converting an acoustic signal into a text transcription. Modern systems achieve this by extracting acoustic features such as Mel spectrograms, from audio and processing them with neural networks. One of the most common and standard metrics to evaluate these systems is with “Word Error Rate (WER)”. It calculates the minimum number of substitutions, deletions and insertions necessary to transform the hypothesis text into the reference text, divided by the number of words in the reference.

Modern ASR systems are mainly based on three different architectural paradigms, each with different implications for how language is modeled. However, the scope of this work is limited to the following two:

Connectionist Temporal Classification Models (CTC): Systems like Wav2Vec2 and Parakeet operate by using an encoder that looks at each individual audio frame and directly assigns it a probability. Essentially, it generates a continuous sequence of guesses for what character from our vocabulary is being spoken at any given moment. The main hurdle here is figuring out how to align the long, variable-length audio input (T frames) with the much shorter, variable-length text output (U characters, where usually $T \gg U$).

To solve this smoothly, the CTC loss function (Graves et al., 2006) essentially evaluates every possible forward-moving way to align the two. A clever trick it employs to make this work is the introduction of a special *blank* token (ϵ). The model can simply output this token at any frame where it is not entirely sure or where no distinct character is being articulated. It’s also used to distinguish between double letters, such as the Ls in the word “call”.

The fundamental mathematical assumption that makes all of this possible is conditional independence. It assumes that the prediction at a specific time step t depends exclusively on the audio information processed at that exact moment, without relying on the surrounding outputs. Formally, this is expressed as $P(y_t | \mathbf{x}) = P(y_t | \mathbf{h}_t)$, meaning that the probability of the output relies solely on the encoder’s internal hidden state $\mathbf{h}_t$ for that specific frame, given the complete input sequence $\mathbf{x}$.

During the final decoding phase, cleaning up the text is quite straightforward. We merely collapse any consecutive repeated characters into a single one and completely eliminate

the blank tokens. So, for example, if the model spits out a raw, unprocessed sequence that looks like $[a, a, \varepsilon, b, \varepsilon, \varepsilon, c]$ simply becomes $[a, b, c]$.

The assumption of conditional independence has a crucial implication for this work: given that each output token is predicted independently and doesn't depend on its context (it is conditioned only on the acoustic characteristics), CTC models have a less explicit relationship to its context. This is not to say that CTC models can't still have an implicit language model, but the encoder-only nature of it may help it be less prone to auto-correction. CTC models may still be affected by its context because the encoder, through training and complex layer techniques, learns to map or otherwise alter the acoustic signal in such a way that it learns language patterns. In essence, a grammatical error may still be corrected in a complex CTC model because the encoder, after learning certain language patterns, sends an output to the CTC layer that forces it to output correct grammar. On the other hand, autoregressive decoders have a much more explicit relationship to context because each token is conditioned explicitly by all the previously generated tokens. This makes CTC models, in theory, generate more "phonetically faithful" transcriptions without trying to restructure them to a more "grammatically correct" output.

Within the CTC paradigm, different architectures handle the acoustic encoding, and the quality of the encoder's representations is critical since CTC places the entire burden of transcription on frame-level classification:

- **Wav2Vec2** (Baevski et al., 2020) relies, fundamentally, on a self-supervised pretraining strategy. Instead of needing perfectly transcribed data from the beginning, the model essentially learns by hiding parts of the audio and attempting to guess what is missing. During this phase, it first passes the raw sound through a convolutional feature extractor to capture the basic acoustic features, and then a standard Transformer encoder takes over to process the sequence. In practice, a random portion of the internal data is intentionally masked. The model is then challenged to pick out the correct missing sound piece from a group of incorrect options or distractors. By doing this repeatedly, it manages to learn highly detailed acoustic representations without needing a single line of human-transcribed text. Once it has been pretrained on massive quantities of unlabeled audio in this manner, the model is finally fine-tuned using actual labeled data, typically incorporating a standard CTC layer to handle the final speech recognition task. This self-taught approach is quite remarkable because it permits Wav2Vec2 to achieve very competitive results even when we have a limited amount of labeled training data. However, it is important to mention that its performance does drop more noticeably when it encounters

speech that falls out of its usual domain, such as the varied accents of second-language (L2) learners or new languages.

- **Parakeet CTC** is built upon a FastConformer architecture (Rekesh et al., 2023), which essentially mixes convolutional layers with self-attention mechanisms inside each block. The local convolutions are in charge of capturing the fine-grained acoustic details within a small window of time, while the self-attention layers take a step back to observe the global dependencies across the entire spoken audio. This hybrid design ends up being highly effective because it permits the model to take advantage of the natural strengths of convolutions, such as focusing on localized sound patterns, as well as the flexibility of attention for the longer-range context. This yields internal representations that are richer than what we would typically get from pure Transformer encoders of a similar size. This design ultimately sets up an implicit language model because the outputs to the CTC layer become conditioned by the other frames in the audio.

The FastConformer variant introduces a few structural optimizations to ease the computational burden. Through efficient data subsampling tricks and streamlined attention processes, it reduces the processing cost barely hurting the overall accuracy. When this architecture is scaled up with an enormous volume of supervised data (reaching 64,000 hours for the 1.1-billion parameter version), it manages to achieve state-of-the-art results for speech recognition, and it does so without the need for a traditional, step-by-step autoregressive decoder.

- **Parakeet TDT-CTC 110M** takes the usual CTC approach and expands it by incorporating what is known as the Token-and-Duration Transducer (TDT) framework (Xu et al., 2023). While standard CTC models are forced to emit exactly one token (or a blank) for every single audio frame, TDT does things a bit differently. It predicts both a token and its duration, basically indicating how many frames to skip forward before making the next prediction. Relaxing this strict, frame-by-frame requirement permits the model to accommodate varying speaking rates in a much more natural way. It also helps to cut down on the excessive number of blank emissions, which in the end can translate to a higher quality of alignment between the raw speech and the text. This 110M version shares the same FastConformer base as the massive 1.1B variant, but employs a much smaller encoder. It has been trained on roughly 36,000 hours of English speech.

Sequence-to-Sequence (Seq2Seq) Models: Systems like OpenAI’s Whisper (Radford et al., 2023) use an encoder-decoder architecture based entirely on Transformers. In a first step, the encoder takes the raw audio, extracts the basic acoustic features over small time windows, and passes them through a series of Transformer blocks. By using self-

attention mechanisms, it evaluates how different sounds relate to one another. Ultimately, this produces a sequence of rich contextual representations (denoted by $\mathbf{h}_1, \dots, \mathbf{h}_T$) which effectively capture the acoustic reality of what was spoken.

Then, the decoder comes into play. It works in an autoregressive manner, which simply means it generates the transcription step-by-step. At any given moment t , it attempts to predict the next token y_t by considering two main sources of information: the history of words it has already produced ($y_1, \dots, y_{t-1}$) and the acoustic evidence provided by the encoder. It achieves this through two types of attention. First, **masked self-attention** allows the model to look back at the sentence it’s building, maintaining a coherent grammatical context. At the same time, **cross-attention** allows it to constantly consult the original audio representations. That means that the probability of the next token depends on both the full acoustic evidence and the entire text generated so far, which can be formally expressed as $P(y_t \mid y_1, \dots, y_{t-1}, \mathbf{h}_1, \dots, \mathbf{h}_T)$.

Because of this step-by-step nature, the decoder fundamentally acts as a **conditional language model**. During its training, it learns the statistical probabilities of how tokens transition from one to another based on massive amounts of text. In the case of Whisper, the model was exposed to approximately 680,000 hours of internet data, where transcripts tend to be quite clean and standardized. As a result, the decoder heavily internalizes the rules of well-formed, grammatically correct language. It learns to naturally expect these proper patterns, from local rules like matching an article with a noun, to longer-range dependencies across the entire sentence.

During the final inference stage, the system uses search strategies (like beam search) to evaluate multiple possible sentence paths in parallel, systematically favoring those that maximize the total probability. Here is where an interesting situation arises for non-native speech. When the acoustic signal is ambiguous, which happens frequently with second-language (L2), the decoder resolves the doubt by defaulting to the most linguistically probable continuation. This introduces a strong tendency to “normalize” or correct the speaker’s errors. For example, if an L2 learner says “Yesterday, I go”, the model might transcribe it as “Yesterday, I went”. It does not do this because the audio clearly sounds like “went”, but rather because the internal language model overwhelmingly dictates that the past tense must follow the context of “yesterday”.

2.2. L2 Learner Speech

The speech of second-language (L2) learners presents distinctive phenomena at multiple levels:

- **Grammatical level:** Agreement errors, incorrect preposition selection, wrong verb forms, and omitted or unnecessary articles, among others.
- **Speech level:** Disfluencies (repetitions, false starts), partial words (abandoned mid-articulation), and pronunciation variants heavily influenced by their native language.

The *Speak & Improve* (S&I) corpus provides word-level annotations that distinguish these phenomena, along with Grammatical Error Correction (GEC) transcriptions. This allows both levels to be studied independently. This work’s main focus is on the grammatical level.

2.3. Computer-Assisted Language Learning (CALL)

Computer-Assisted Language Learning (CALL) refers to the use of technology to support and enhance language instruction (Eskenazi, 2009). In CALL systems that incorporate spoken feedback, ASR plays a central role: the accuracy of the transcription and, the preservation of learner errors determine whether the system can deliver pedagogically useful feedback. A transcription that silently corrects a learner’s grammatical mistakes provides no opportunity for error-based instruction, making error preservation extremely important for ASR components in CALL pipelines.

2.4. ERRANT: Grammatical Error Taxonomy

ERRANT (*ERRor ANnotation Toolkit*) is a tool that automatically classifies edits between a source text and a target text into linguistic categories. Given a pair (original text, corrected text), ERRANT identifies edits by categorizing them into three main families:

- **Missing-word errors (M:)** — An expected word is absent (e.g., M:DET for missing determiners like “I have ... dog”).
- **Replacement errors (R:)** — An incorrect word or form is used instead of the correct one (e.g., R:PREP for preposition replacements like “arrive **to**” → “arrive **at**”, or R:VERB:FORM for “travelling” → “travel”).
- **Unnecessary-word errors (U:)** — An extra word is present that should be removed (e.g., U:DET for “the most of people” → “most of people”).

In the *eval* set of the S&I corpus, ERRANT initially identified 13,783 raw grammatical errors distributed across 47 distinct error types that naturally occurred in the learners’ speech. However, not all written grammatical errors are meaningful for evaluating ASR systems on **spoken** L2 grammar.

To ensure the Grammatical Error Preservation (GEP) metric accurately reflects relevant spoken grammar auto-corrections, we applied a strict filter to discard error types that

introduce noise or are inherently tied to written language artifacts. Eight ERRANT categories were excluded:

Type	Reason for exclusion
R:SPELL, R:ORTH	Orthographic edits; ASR word mismatches belong in WER, not grammatical preservation. For instance, if an ASR outputs “addictives” instead of “addictors”, it is acoustically correct even if written differently.
M:PUNCT, U:PUNCT	Punctuation; CTC backends do not emit punctuation natively, so cross-model comparison is unfair and meaningless for spoken language.
M:OTHER, R:OTHER, U:OTHER	ERRANT catch-alls for multi-word paraphrases or stylistic rewrites; they do not model single grammatical errors and artificially inflate mutation rates due to their length.
R:WO	Word-order swaps; edit-distance alignment cannot classify corrected vs. preserved reliably (the correction rate is effectively zero).

After removing these 3,386 edits (24.6% of the dataset), we retained **39 informative ERRANT types**, comprising a total of 10,397 clean, single-word grammatical errors directly meaningful for L2-speaking feedback. This filtered taxonomy enables a precise and granular analysis of which types of grammatical errors are preserved or corrected by each ASR model.

3. Data and Resources

3.1. Speak & Improve Corpus

The *Speak & Improve* (S&I) corpus from the SANDi 2025 challenge contains audio recordings of English learners with varying proficiency levels and native languages (Knill et al., 2024; Qian et al., 2024). Three distinct partitions are utilized in this work to cover both baseline evaluation and fine-tuning experiments:

Partition	Utterances	Ref. Words	Speech Phenomena	Raw Gram. Errors	GEP-scored Errors
Train	3,883	170,417	10,849	8,888	6,490
Dev	3,249	133,558	8,408	92	64
Eval	3,209	131,615	8,614	13,783	10,397
Total	10,341	435,590	27,871	22,763	16,951

The *eval* partition contains significantly more GEC (Grammatical Error Correction) annotations than the *dev* partition (13,783 raw edits vs. 92), making it the primary source for statistically significant baseline results. The *train* partition is used exclusively for training the fine-tuned models, providing 6,490 GEP-scored grammatical errors across 1,919 GEC-annotated utterances. The *dev* partition is omitted from results sections due to this severe annotation sparsity (only 64 scored errors remain after the same filtering), which prevents any robust quantitative cross-validation of grammatical error preservation.

The 24.6% of raw edits that are excluded from GEP scoring (e.g., 3,386 edits in the *eval* set, leaving 10,397 scored errors) are discarded based on the strict taxonomy filters described in Section 2.4. These represent written-language orthographic conventions (R:SPELL, R:ORTH), punctuation marks (M:PUNCT, U:PUNCT), or multi-word catch-all paraphrases (*:OTHER) and word-order swaps (R:W0) that cannot be reliably assessed by simple token-level edit-distance alignments. Excluding them ensures the GEP metric is highly focused on authentic, spoken grammatical error preservation. During both training and evaluation, these excluded categories are completely omitted and ignored. They are not treated as grammatical errors that should be preserved, nor do they impact the model’s GEP or mutation rate if the model changes them. This ensures that only the 39 informative spoken-grammar categories are evaluated.

The corpus provides two parallel transcriptions for each utterance:

File	Content
trans-ref.json	Verbatim transcription containing word-level annotations (disfluency marks, pronunciation variants, and partial words).
gec-ref.json	Grammatically corrected transcription (GEC) representing the target standard English fluent text.

While the raw annotations in `trans-ref.json` contain valuable acoustic markers (such as disfluency marks, pronunciation variants, and partial words), they must be stripped beforehand to construct a clean, fluent verbatim transcription for Grammatical Error Preservation (GEP) evaluation. This preprocessing is essential because raw speech disfluencies and abandoned fragments would otherwise corrupt the token-level alignment and classification processes in downstream text-processing and parsing tools.

Furthermore, the target standard English text in `gec-ref.json` does not contain any pre-annotated grammatical error labels, error spans, or error tags. The S&I corpus only provides parallel verbatim and corrected transcriptions without metadata indicating where or what the errors are. This absence of direct labels necessitates using the ERRANT alignment toolkit: by dynamically comparing the preprocessed, fluent verbatim transcription against the corrected GEC transcription, ERRANT pinpoints the exact location, edit operation, and grammatical category of each speaker error to generate the GEP targets used throughout this work.

Additionally, the corpus includes **SLA scores** (*Spoken Language Assessment*) per exam session: a continuous score on a scale from 2.0 to 6.0 assigned by human examiners reflecting the speaker’s overall oral proficiency level. These scores are distributed in TSV files per partition (`eval-sla-overall.tsv`, etc.) and allow the stratification of results by proficiency level.

4. Benchmark Methodology

This chapter describes how a raw S&I partition is turned into the WER, EPR and GEP numbers. The same *pipeline* is used for the baseline comparison in Section 6, the light-weight-model evaluation in Section 7, and the DPO-fine-tuned checkpoint in Section 8, so this section is the single source of metric definitions for the rest of the work.

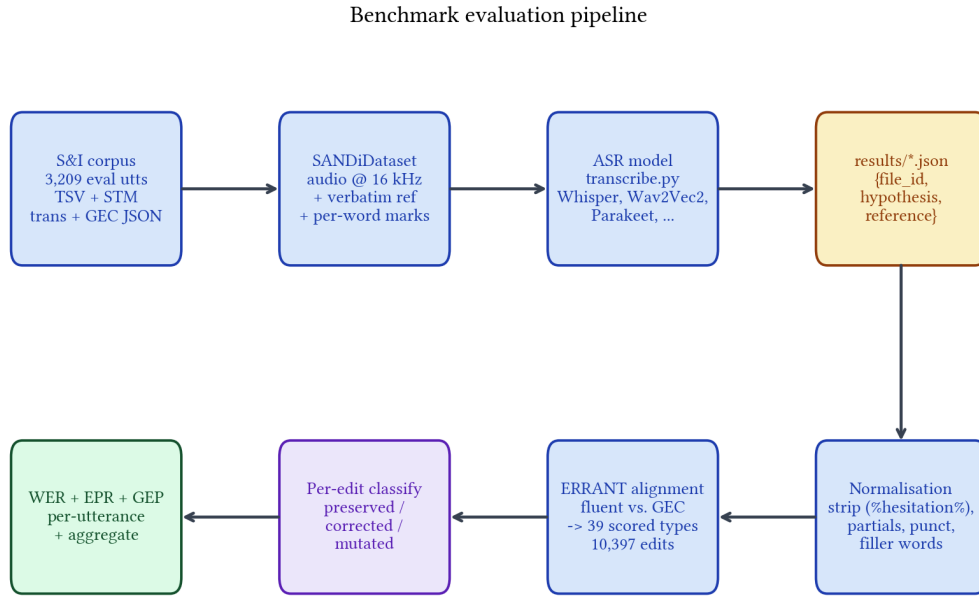


Figure 1: Benchmark evaluation pipeline. The S&I corpus is loaded together with its STM verbatim, per-word annotations and GEC reference; audio is transcribed by each ASR model and stored as JSON; the reference and hypothesis are then normalized, aligned, and scored by three complementary metrics. Boxes are colored by stage type: blue for data and processing steps, yellow for the intermediate hypothesis artifact, purple for the per-edit classification, green for the final metric output.

The pipeline of Figure 1 has two purposes that are easy to confuse and important to keep separate. The first half (the top row) is purely about getting an ASR hypothesis for every audio file: load the corpus, run the model, write the result to disk. Nothing model-specific or grammar-aware happens here. The second half (the bottom row) is where the actual evaluation lives: a single shared normalization step feeds three different scoring procedures, two classic (WER, EPR) and one novel (GEP), each of which answers a different question about the same hypothesis. Splitting the pipeline this way means that re-running a model is independent from re-running the metrics. Whenever the GEP definition is refined (for example when the ERRANT exclusion list is updated, as described in Section 2.4), every model’s `*-eval.json` is regenerated from cached transcriptions without ever touching the GPU again.

4.1. Loading the Speak & Improve Corpus

The S&I corpus is distributed as four parallel artifacts per partition. The audio itself is referenced indirectly by a TSV *file list* (one `file_id` and one relative audio path per line); the STM file contains the time-aligned verbatim reference; the `trans-ref.json` annotation file carries per-word marks (disfluency, pronunciation, partial) and structural tags (hesitation); and the `gec-ref.json` file provides the grammatically corrected version of the same utterance. The exact partition sizes are summarized in the table in Section 3.1.

The first stage of the pipeline fuses these four artifacts into a single in-memory object. The `SANDiDataset` class in `src/dataset.py` reads the TSV, attaches the STM reference and segment boundaries to each `Utterance`, and lazily loads the JSON annotations when a downstream step asks for them. Each utterance is keyed by its `file_id`, which is the only identifier that flows through the rest of the pipeline.

This design has one practical consequence worth noting. The four input files were produced by different annotation passes and are not always strictly consistent: the STM text and the JSON Transcript list represent the *same* spoken words but with slightly different conventions (the STM contains the parenthesised marker strings `(%hesitation%)` and `(univers-)` inline; the JSON splits them into dedicated `tag` or `marks` entries). The benchmark needs both: WER will score against the STM string, EPR will iterate over the JSON marks, and GEP will rebuild a fluent verbatim from the JSON words. Loading them together once, into the same `Utterance`, keeps the three views aligned by `file_id` and avoids any per-metric resynchronisation.

4.2. ASR Transcription

The second stage iterates over the loaded dataset and produces one hypothesis per utterance. The audio is read with `soundfile`, downmixed to mono and resampled to 16 kHz with `librosa` (McFee et al., 2015) (the sample rate all evaluated models expect), and then handed to a uniform `ASRModel.transcribe(audio, sr) -> str` interface defined in `src/models.py`. This wrapper hides the differences between the Whisper, Wav2Vec2, NeMo and Moonshine backends: each backend takes care of its own tokenizer, decoding strategy and chunk-stitching internally, and returns a single string.

The hypothesis is stored *verbatim* as the model emits it, including casing, punctuation and any filler tokens the model chose to insert. This is a deliberate choice: keeping the raw output preserves all the information needed by downstream EPR alignment (where word-level identity matters) and avoids any model-specific normalization creeping into the evaluator. The transcribed results are written to `results/<model>/<split>.json` with one

entry per utterance, each containing the `file_id`, the `hypothesis`, the original reference, the `audio_duration_s`, and the `inference_time_s` used to compute the real-time factor (how quickly it inferences in relation to audio duration). From this point on, no model is ever loaded again: every subsequent step in the pipeline operates on these JSON files alone.

4.3. Normalization Before Scoring

Before the reference and the hypothesis can be compared, they have to be brought into a common surface form. This is the first big design decision of the pipeline, and it has direct consequences for what the three metrics measure. Two normalizers are applied, one to the reference and one to the hypothesis, both implemented in `src/evaluate.py`.

The reference normalizer `normalize_ref` performs four operations in order: it strips (`%hesitation%`) markers (which are structural placeholders, not spoken words), strips (`(partial-)`) fragments (incomplete words the speaker abandoned mid-articulation), lowercases the text, and removes punctuation. The hypothesis normalizer `normalize_hyp` lowercases, removes punctuation, and drops a fixed list of filler tokens (`uh`, `um`, `er`, `hmm`, `mm`, ...). All four marker types and all the filler tokens are treated as *optionally deletable* by the official S&I scoring guidelines, so the two normalizers are simply enforcing the same convention on both sides of the comparison.

A concrete example makes this concrete. Suppose the STM reference is

"She (`%hesitation%`) have many friend (`(univers-)`)"

and the Whisper hypothesis is

"She has many friend, `um`, university."

After normalization both become candidates the WER scorer can align. The reference becomes "she have many friend" and the hypothesis becomes "she has many friend university" — the hesitation marker, the partial, the punctuation, the casing and the filler "um" are all gone, and the only differences that remain are real lexical differences.

It is important to stress what normalization throws away and what is kept track of elsewhere. The four marker types stripped from the reference (`(%hesitation%)`, `(partial-)`, plus their JSON counterparts `disfluency` and `pronunciation`) are exactly the speech-production phenomena that EPR scores in Section 4.5; they are removed *from the WER reference* so that no model is unfairly penalized for failing to emit a (`(univers-)`-style fragment it could never produce, but they are not forgotten. Likewise the grammatical errors that motivate this work live in the surviving lexical content (`have` instead of `has`, `friend` instead of `friends`), so they pass through the WER normalizer untouched and are the subject of GEP scoring in Section 4.6.

4.4. Word Error Rate

The first metric is the classical Word Error Rate, computed by `compute_wer_metrics` in `src/evaluate.py` via `jiwer.process_words`¹. After normalization, the reference and the hypothesis are both word sequences; an edit-distance alignment between them yields the minimum number of substitutions S , deletions D and insertions I needed to turn the hypothesis into the reference, and WER (Morris et al., 2004) is the ratio of these edits to the number of reference words N :

$$\text{WER} = \frac{S + D + I}{N}$$

Concretely, suppose the normalized reference is "i go to school yesterday" (5 words) and the normalized hypothesis is "i went school in yesterday" (5 words). The optimal alignment introduces six positions — one per word in the reference plus an extra slot for the inserted word in the hypothesis:

	pos. 1	pos. 2	pos. 3	pos. 4	pos. 5	pos. 6
ref	i	go	to	school	–	yesterday
hyp	i	went	–	school	in	yesterday
op	equal	substitute	delete	equal	insert	equal

The double dash marks a position where the alignment expected a word and found none. Counting the operations gives one substitution ($\text{go} \rightarrow \text{went}$), one deletion (to), one insertion (in) and three equal matches, so $S = 1$, $D = 1$, $I = 1$ against $N = 5$ reference words, yielding $\text{WER} = \frac{3}{5} = 60\%$. The pipeline computes this both as an overall figure across the partition (using the totals of S , D , I and N) and per utterance.

WER is necessary but not sufficient for the question this work asks. It counts edits but does not distinguish *what kind* of edit the model made. If the reference says "i go yesterday" and the model emits "i went yesterday", WER records exactly one substitution: it cannot tell the reader whether the model silently corrected the learner's tense error (which is the auto-correction phenomenon the work is about) or simply mis-transcribed the verb (which is unrelated transcription noise). A model with the same WER as another can be qualitatively very different in how it treats learner errors. That gap is precisely what the next two metrics fill.

4.5. Error Preservation Rate (EPR)

EPR is the speech-phenomena counterpart of GEP: it tracks the fate of the annotation marks that WER explicitly throws away. For each word in the verbatim that the JSON annota-

¹<https://github.com/jitsi/jiwer>

tion flags with at least one mark (disfluency, pronunciation, partial), the question is whether the ASR hypothesis still has that word at the corresponding position.

The implementation in `compute_epr` works in two steps. First, the full verbatim word list (including the marked words) is aligned against the normalized hypothesis with `jiwer.process_words`, yielding for every reference position one of three alignment types: `equal` (the hypothesis has the same word at the aligned position), `substitute` (the hypothesis has a different word at the aligned position) or `delete` (the hypothesis has nothing at all at the aligned position). For each marked word, the alignment type is read off, the word is recorded as *preserved* / *substituted* / *deleted* accordingly, and the count is incremented under that mark type.

For a partial-word example, suppose the verbatim contains the fragment "i (univers-) i went to university" (with (univers-) carrying the JSON mark `partial`). If the ASR emits "i univers i went to university" the partial slot aligns as `equal` and the mark is counted as *preserved*; if it emits "i i went to university" the slot aligns as `delete` and the mark is counted as *deleted*; if it emits "i universe i went to university" the slot aligns as `substitute` and the mark is counted as *substituted*.

EPR is reported in this work only as a supporting metric inside the DPO results table in Section 8 (where it sanity-checks that the fine-tuned model has not become more aggressive at smoothing disfluencies away). The central comparison between architectures uses WER and GEP.

4.6. Grammatical Error Preservation (GEP)

GEP is the metric this work introduces. The intuition is simple: a learner’s spoken utterance contains grammatical errors that a tutoring application needs to see; an ASR model can either reproduce those errors faithfully (good for tutoring), silently restore them to standard English (bad for tutoring), or mis-transcribe them altogether (noise). GEP scores each individual grammatical error in the *eval* partition into one of these three buckets: *preserved*, *corrected*, and *mutated*.

The implementation lives in `compute_gep` in `src/evaluate.py` and proceeds in three stages: build the per-utterance list of grammatical errors using ERRANT, align the fluent verbatim against the ASR hypothesis, and then classify each error from the alignment.

4.6.1. Building the Targets

The S&I corpus does not ship pre-annotated grammatical-error labels. What it does ship is two parallel transcripts per utterance: the verbatim `trans-ref.json` (what the learner actually said) and the GEC `gec-ref.json` (the same content in standard English). The

grammatical errors are implicit in the difference between the two, and ERRANT (*ERRor ANnotation Toolkit*, Bryant et al., 2017) is the tool that makes them explicit.

The verbatim transcript first has to be cleaned. Words tagged *disfluency* or *partial* in the JSON annotation are removed by `_build_fluent_words`, producing a *fluent verbatim* that contains only the words the speaker actually completed and that belong to the grammatical content. Pronunciation marks are kept (they label real, complete words that simply sound off). The fluent verbatim and the GEC text are then parsed and aligned by ERRANT, which returns a list of edits, each one tagged with a type (R:VERB:SVA, M:DET, ...), an erroneous span `o_str` at positions `[o_start, o_end)` in the fluent verbatim, and a corrected span `c_str`. The edit types follow the three-family taxonomy already described in Section 2.4: *Replacement* (R:xxx), *Missing-word* (M:xxx) and *Unnecessary-word* (U:xxx).

Not every edit ERRANT returns is meaningful for evaluating spoken grammar. The eight categories listed in Section 2.4 (R:SPELL, R:ORTH, M:PUNCT, U:PUNCT, M:OTHER, R:OTHER, U:OTHER, R:W0) are removed by the `GEP_EXCLUDED_TYPES` filter, leaving 39 informative ERRANT types and, in the eval partition, exactly 10,397 scored errors. These are the targets that GEP scoring operates on.

4.6.2. Classifying Each Edit

Once the targets are known, the ASR hypothesis enters the picture. The fluent verbatim is aligned to the normalized hypothesis with `jiwer.process_words` once per utterance, producing the same kind of position-to-position alignment used by EPR. Each surviving ERRANT edit is then classified into *preserved*, *corrected* or *mutated* depending on what the hypothesis says at the relevant positions.

The three ERRANT edit families need three slightly different rules:

- **Replacement edits** (R:xxx) are the core case. The hypothesis words aligned to fluent positions `[o_start, o_end)` are collected into a span. If that span equals `o_str` (the erroneous form), the edit is *preserved*: the model emitted the same wrong word the learner did. If it equals `c_str` (the corrected form), the edit is *corrected*: the model silently produced the standard form. Anything else is *mutated*: the model emitted something that matches neither the error nor the correction.
- **Unnecessary-word edits** (U:xxx) have a non-empty erroneous span and an empty corrected span (the word should not be there at all). The same hypothesis span is collected. If it equals `o_str` the edit is *preserved* (the unnecessary word is still there in the hypothesis); if the hypothesis emitted nothing at all at that slot the edit is *corrected* (the model dropped the unnecessary word); otherwise it is *mutated*.

- **Missing-word edits** (M:xxx) are the trickiest case because they have zero span in the fluent verbatim: the error is *the absence of a word*. There is no fluent position to read off, so the heuristic in `_hyp_has_insertion_near` looks instead at the hypothesis *between* the words aligned to fluent positions `o_start - 1` and `o_start`, and reports the edit as corrected if the missing word from `c_str` is found in that gap (the model inserted what the learner omitted) and as preserved otherwise.

The same per-edit decision is also recorded with the actual ASR span the model produced, so that the per-utterance `*-eval.json` keeps a full audit trail and downstream analyses can drill into individual cases.

4.6.3. Three States, Not Two

The simplest possible version of this metric would only have two outcomes: the model either preserved the error or it did not. The three-way split is what makes the metric useful, and the gap between *corrected* and *mutated* carries the central architectural claim of this work.

Consider a learner who says "yesterday i go to school". Two different ASR systems might both fail to preserve the tense error:

- A Whisper-class Seq2Seq model emits "yesterday I went to school." — the autoregressive decoder has produced the standard form because its language model strongly prefers *went* after *yesterday*. This is an *intentional* rewrite. From a tutoring point of view, the diagnostic signal has been destroyed.
- A Wav2Vec2 model emits "yestday i goh to schol" — the encoder has simply misheard most of the utterance. This is *transcription noise*. From a tutoring point of view, the signal is also destroyed, but for a completely different reason: the model is not *correcting* the learner, it is just bad.

A two-way preserved-vs-not-preserved split would lump these two failures together. With three states, the first case is *corrected* and the second is *mutated*, and the per-model breakdown of corrected / mutated shares (visible in Figure 6 and Figure 8) is what lets the work claim that Seq2Seq decoders are biased *towards the standard form*.

4.7. A Worked End-to-End Example

To make the full pipeline tangible, consider a single synthetic utterance traced through every stage.

The STM reference is

"she (%hesitation%) have many friend (univers-)"

and the JSON annotation for the same utterance contains one structural hesitation tag, the words she, have, many, friend with no marks, and one final word UNIVERS carrying the mark partial. The GEC reference is

"she has many friends".

For **WER**, `normalize_ref` strips the hesitation marker, the partial, the case and punctuation to give "she have many friend" (4 words). Two ASR hypotheses are scored:

- Whisper emits "She has many friend." — after `normalize_hyp` this becomes "she has many friend". The alignment marks one substitution at position 1 (have → has), so the per-utterance WER is $\frac{1}{4} = 25\%$.
- Wav2Vec2 emits "she av many frend" — already lowercase and clean. The alignment marks two substitutions (have → av, friend → frend), so the per-utterance WER is $\frac{2}{4} = 50\%$.

For **EPR**, the full verbatim word list (including the partial-marked UNIVERS) is aligned against each normalized hypothesis. Neither hypothesis contains anything resembling UNIVERS after the last friend, so in both cases the partial-marked slot aligns as delete and the single mark of type partial is counted as deleted. Neither ASR preserved this disfluency.

For **GEP**, `_build_fluent_words` constructs the fluent verbatim by dropping the partial-marked UNIVERS, giving the same string as the WER reference: "she have many friend". ERRANT compares this against the GEC "she has many friends" and returns two edits, both of which survive the `GEP_EXCLUDED_TYPES` filter:

Type	Fluent span	o_str	c_str
R:VERB:SVA	pos. 1	have	has
R:NOUN:NUM	pos. 3	friend	friends

The classifier then aligns the fluent verbatim against each hypothesis and reads off the ASR span at the two edit positions:

Model	Edit	ASR span	Comparison	Status
Whisper	R:VERB:SVA	has	matches c_str	corrected
Whisper	R:NOUN:NUM	friend	matches o_str	preserved
Wav2Vec2	R:VERB:SVA	av	matches neither	mutated
Wav2Vec2	R:NOUN:NUM	frend	matches neither	mutated

Notice what WER and GEP disagree about. At the R:VERB:SVA site, both models contribute exactly one substitution to WER — WER alone cannot tell them apart. GEP separates them sharply: Whisper *intentionally produced the standard form* (corrected) while Wav2Vec2

produced noise (mutated). This is the per-edit-level evidence that aggregates, across 10,397 such edits, into the architectural conclusions of Section 6.

4.8. Aggregation and Outputs

The output of the per-utterance scoring is a single `results/<model>/<split>-eval.json` file containing the WER metrics (total S , D , I , N and the per-utterance list), the EPR breakdown by mark type with preserved / deleted / substituted counts, and the GEP breakdown by ERRANT type with preserved / corrected / mutated counts plus a per-utterance audit trail of every classified edit. Aggregate figures (the overall preservation rate, the overall correction rate, the overall mutation rate) are computed on the totals of these counters.

These per-utterance JSON files are the single data source that every downstream analysis in the thesis reads from. The model comparison in Section 6 simply loads one file per model and plots their summary statistics; the per-ERRANT-type heatmaps in Figure 7 and Figure 8 read the same per-type counts; the proficiency-band analysis in Section 6.5 joins each utterance's `file_id` against the SLA score table and re-aggregates the same counts within each band; the cross-validation in Figure 12 partitions the utterance list into ten folds and re-aggregates ten times; and the DPO evaluation in Section 8 runs through the exact same pipeline on a different `results/` directory. No model is re-loaded, no audio is re-decoded, and no metric is redefined: every result in the rest of this document is a different slice of these per-utterance JSON files.

5. Evaluated Models

A total of seven ASR models were evaluated across two separate analyses. The primary comparison involves four models representing two fundamentally different architectural paradigms, selected because they provide the clearest contrasts in architecture, scale, and training data:

Model	Architecture	Parameters	Type	Language Model
Whisper Small	Transformer Encoder-Decoder	244M	Seq2Seq	Implicit in decoder
Whisper Medium	Transformer Encoder-Decoder	769M	Seq2Seq	Implicit in decoder
Wav2Vec2 Large	Transformer Encoder + CTC	317M	CTC	Implicit in encoder
Parakeet CTC 1.1B	FastConformer + CTC	1,063M	CTC	Implicit in encoder

Comparing Whisper Small and Medium enables studying the effect of model scale within the same Seq2Seq family. Wav2Vec2 Large and Parakeet CTC 1.1B represent the CTC paradigm at two radically different scales: 317M vs. 1,063M parameters, with different training data (960h of LibriSpeech vs. 64,000h of English speech). This isolates the effect of scale from the effect of architecture.

Additionally, three lightweight models (61–182M parameters) are evaluated in a dedicated section to determine the feasibility of deployment in resource-constrained environments:

Model	Architecture	Parameters	Type	Selection Rationale
Parakeet TDT-CTC 110M	FastConformer + TDT/CTC	114M	Hybrid CTC	Compact version of the best baseline; tests scalability of the CTC architecture
Canary 180M Flash	FastConformer + Transformer dec.	182M	Encoder-Decoder	Multilingual model with a small decoder; tests decoder impact at reduced scale
Moonshine Base	Encoder-Decoder	61M	Encoder-Decoder	Ultra-lightweight model for edge devices; tests the limits of minimal capacity

These lightweight models were chosen to span the spectrum of reduced architectures: a CTC-only model (Parakeet 110M), a small encoder-decoder (Canary), and an ultra-minimal Seq2Seq model (Moonshine). Together they answer whether the CTC advantage observed in the baseline comparison holds at reduced scales.

Working hypothesis: Seq2Seq models with autoregressive decoders (Whisper) will exhibit a greater tendency to correct grammatical errors than CTC models, since the decoder provides a more explicit relationship with its context, as opposed to CTC where the relationship is much more implicit. Among CTC models, Parakeet should offer better WER than Wav2Vec2 (due to its massive data and FastConformer architecture) without increasing grammatical auto-correction.

6. Results

All results reported in this section correspond to the *eval* set (3,209 utterances, 131,615 reference words, 10,397 GEP-scored grammatical errors), unless stated otherwise. The stability and robustness of these evaluation metrics are validated through 10-fold cross-validation on the eval set in Figure 12.

6.1. Word Error Rate (WER)

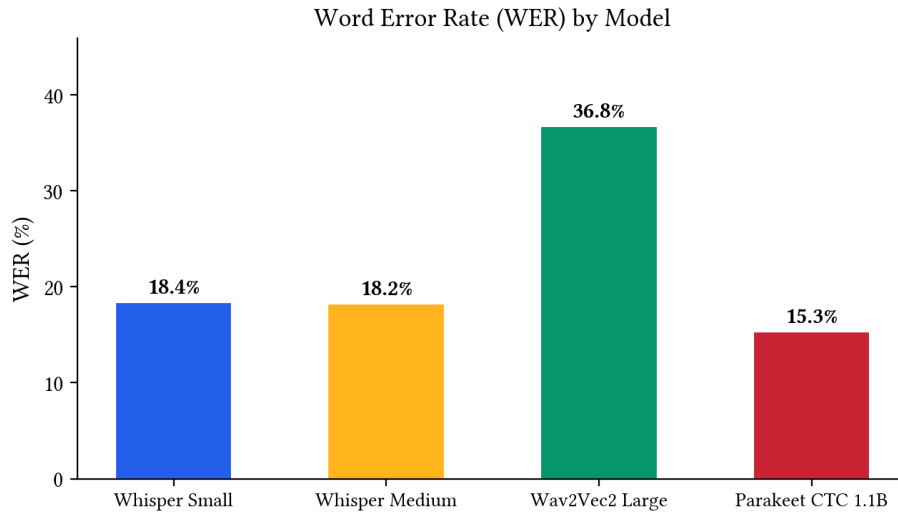


Figure 2: Overall WER comparison among the four evaluated models on the eval set (3,209 utterances).

Parakeet CTC 1.1B achieves the best WER (15.35%), outperforming both Whisper models: Small (18.39%) and Medium (18.23%). Wav2Vec2 Large falls significantly behind (36.76%). This result reveals that Parakeet’s data scale and FastConformer architecture compensate for the absence of a language model, achieving better precision than Seq2Seq models even on L2 speech. The WER values are higher than in conventional native speech benchmarks, reflecting the intrinsic difficulty of learner speech.

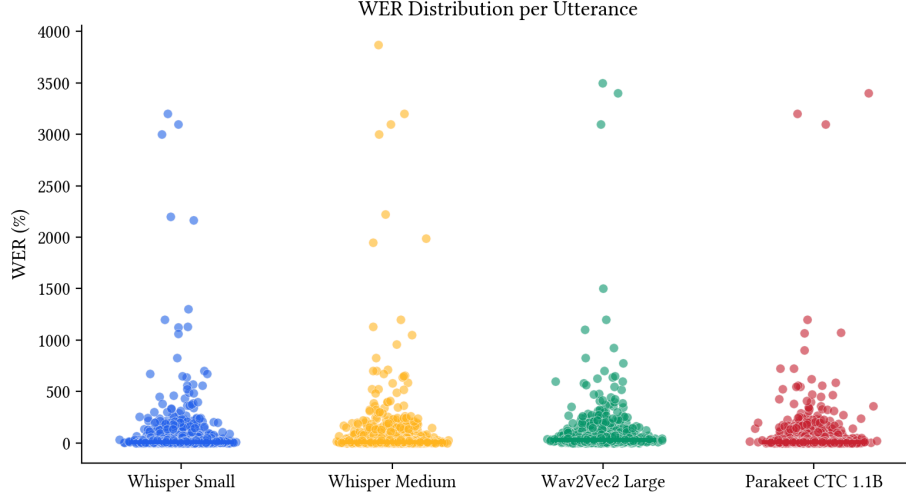


Figure 3: WER distribution per individual utterance (3,209 points per model).

The per-utterance distribution (Figure 3) reveals that all models exhibit high variance, with some particularly problematic utterances ($WER > 100\%$, indicating the model “hallucinated” additional content). Wav2Vec2 shows greater dispersion, reflecting its sensitivity to L2 pronunciation variants.

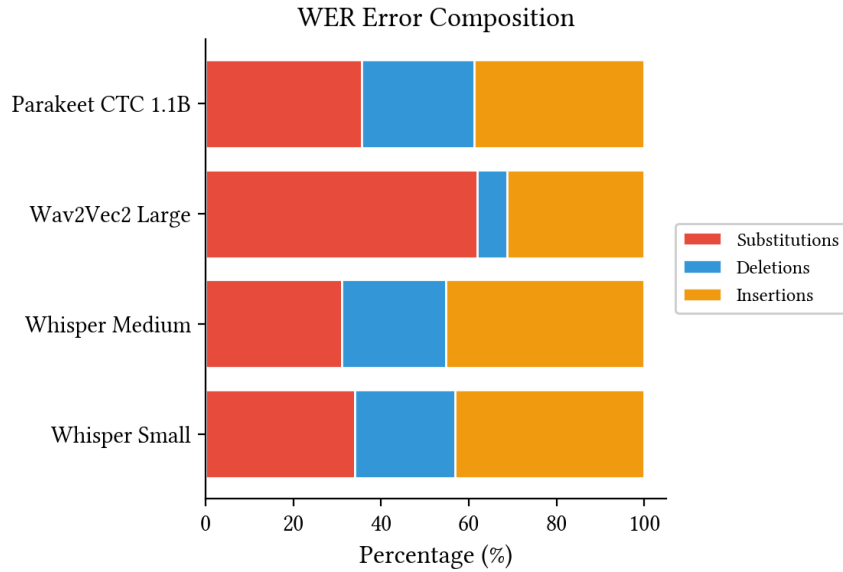


Figure 4: WER error breakdown: substitutions, deletions, and insertions.

The error composition analysis (Figure 4) reveals structural differences: Wav2Vec2 generates predominantly **substitutions** (61.9%). Parakeet CTC, despite also being CTC, shows a more balanced distribution (35.5% substitutions, 25.7% deletions, 38.8% insertions), benefiting from its massive training. Whisper models generate proportionally more insertions (43–44%), suggesting that the decoder tends to “expand” the output.

6.2. Grammatical Error Preservation (GEP)

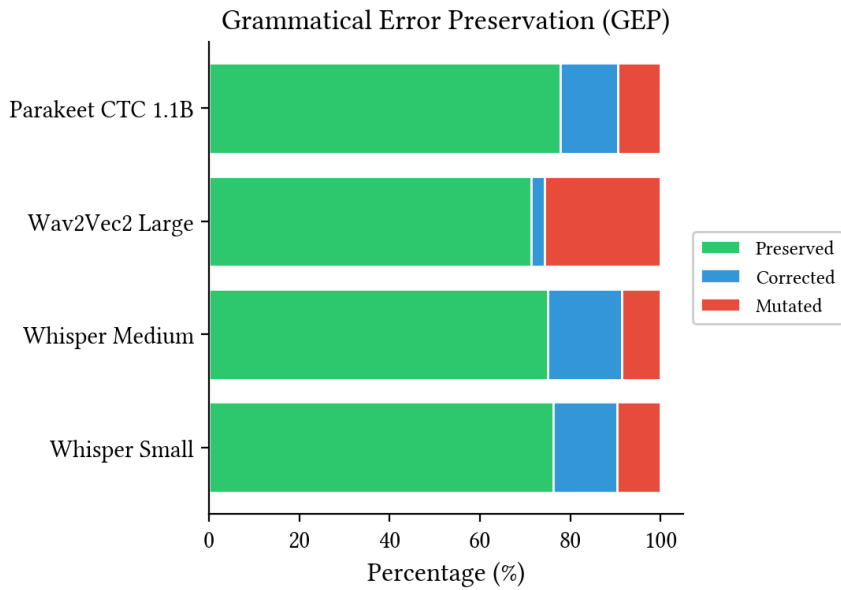


Figure 5: Overall grammatical error preservation: proportion preserved, corrected, and mutated (10,397 scored errors, 39 ERRANT types).

The GEP results (Figure 5) constitute the central finding of this work. After filtering (Section 2.4), there were **10,397 grammatical errors** in the eval set across **39 ERRANT types**, providing a robust statistical foundation focused on spoken L2 grammar.

Parakeet CTC 1.1B preserves 77.7% of grammatical errors, the highest rate among all models, correcting 12.8% and mutating 9.5%. **Whisper Small** preserves 76.3%, correcting 14.1%. **Whisper Medium** preserves 75.0%, but corrects 16.4% (the highest auto-correction rate). **Wav2Vec2** preserves 71.4%, corrects 2.9%, but mutates 25.7%.

Parakeet emerges as an optimal point: better WER than any other model **and** higher grammatical preservation than both Whispers. Its correction rate (12.8%) is lower than Whisper Small’s (14.1%) and Whisper Medium’s (16.4%), contributing to the claim that the CTC architecture is inherently less prone to auto-correction, even at a massive scale.

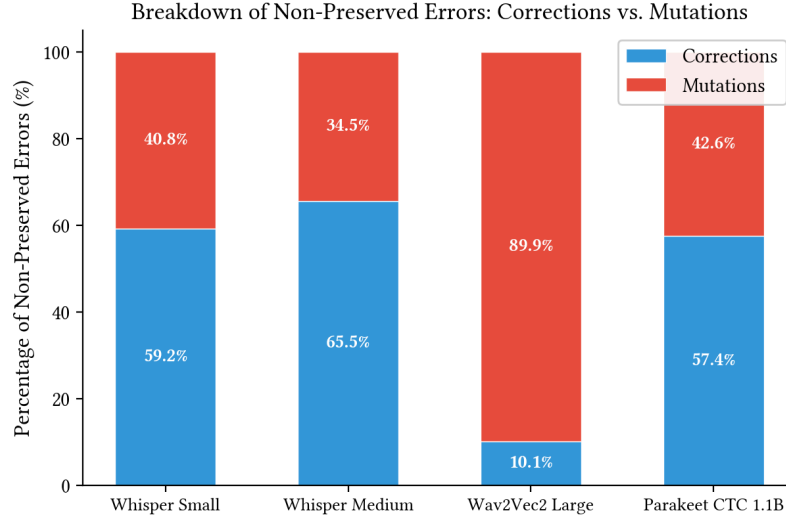


Figure 6: Breakdown of non-preserved grammatical errors into corrections (the model produced the grammatically correct form) and mutations (the model produced a different, unrelated form). This separates intentional auto-correction from transcription noise.

Figure 6 reveals a more nuanced picture than raw correction rates alone. Among the errors that each model **changed** (i.e., did not preserve), the fraction attributable to actual corrections versus random mutations differs dramatically: Whisper Medium corrects 66% of its changed errors, Whisper Small 59%, and Parakeet 57%, while Wav2Vec2 corrects only 10% and the remaining 90% of its non-preserved errors are mutations, not corrections. This serves as evidence that Seq2Seq models are **intentionally** auto-correcting grammatical errors through their language model, whereas Wav2Vec2’s changes are overwhelmingly mutations (90%) rather than corrections, indicating that its low overall correction count reflects more its high overall inaccuracy. Parakeet CTC, despite being a CTC model, shows corrective precision close to Whisper Small (57% vs. 59%), suggesting that its massive training data enables it to learn some degree of linguistic correction even without an autoregressive decoder.

6.2.1. Analysis by ERRANT Error Type

The previous sections report aggregate preservation and correction rates, but different types of grammatical errors behave very differently under ASR. To understand *which* errors are most at risk and *why*, this section examines the 10,397 scored errors across 39 ERRANT types at the per-type level.

Two complementary heatmaps are presented. Figure 7 shows the **preservation rate** per type: the fraction of errors that the ASR model faithfully reproduces. Figure 8 shows the **correction rate** per type: the fraction that the model replaces with the grammatically standard form. The residual difference between 100% and the sum of both rates corresponds to mutations — cases where the model produces a form that matches neither the error

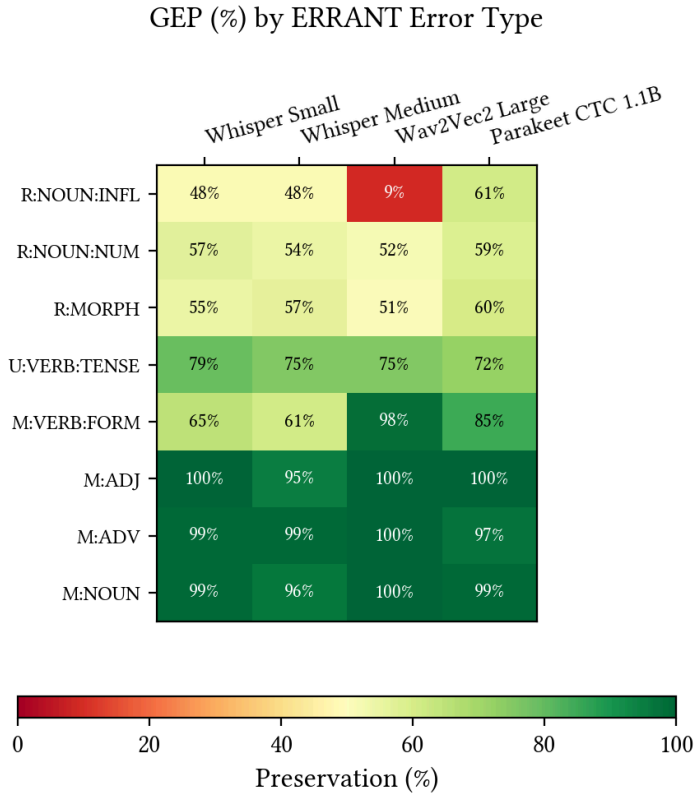


Figure 7: Heatmap: GEP preservation rate by ERRANT error type and model. Only types with at least 30 errors per model are included to ensure statistical robustness. Types selected include the 3 best-preserved, 3 worst-preserved, and the most distinctive type per model (best and worst relative performance).

nor the correction. Reading both heatmaps together reveals whether non-preservation of a given type is driven by intentional auto-correction (high correction rate) or by transcription noise (high mutation rate), a distinction that the preservation heatmap alone cannot make. Figure 9 provides a complementary view of the five most frequent error types.

Three patterns emerge from this analysis, each revealing a different facet of how ASR models interact with learner errors.

6.2.1.1. Morphosyntactic Errors

Errors involving local grammatical rules — number agreement, subject-verb agreement, morphological form — are the types most frequently auto-corrected by all models with sufficient language awareness. These are errors where a small change to one word can be predicted from its immediate neighbours:

- **R:NOUN:NUM** ($n = 1028$, noun number; e.g., “many *student*” → “many *students*”): Whisper Medium corrects 35%, Whisper Small 31%, Parakeet 31%, Wav2Vec2 4%. The preservation rate ranges from 52% (Wav2Vec2) to 59% (Parakeet).
- **R:VERB:SVA** ($n = 459$, subject-verb agreement; e.g., “she *have*” → “she *has*”): Whisper Medium corrects 34%, Whisper Small 31%, Parakeet 22%, Wav2Vec2 3%.

Correction Rate (%) by ERRANT Error Type

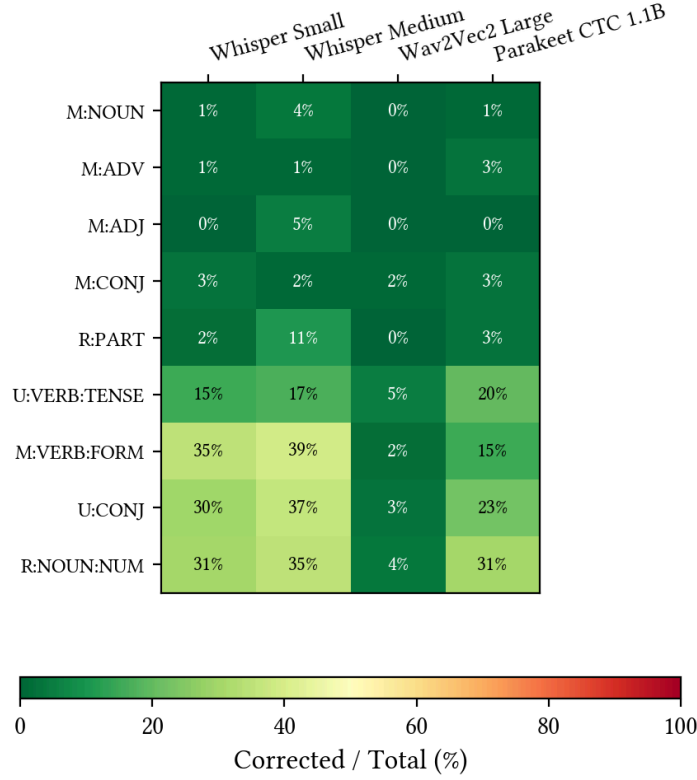


Figure 8: Heatmap: correction rate ($\frac{\text{corrected}}{\text{total}}$) by ERRANT error type and model. This complements Figure 7 by disambiguating whether non-preserved errors are due to intentional auto-correction or random mutation. Only types with at least 30 errors per model are included. Types selected include the 3 highest and 3 lowest average correction rates, plus the most distinctive type per model.

- **R:MORPH** ($n = 401$, morphology; e.g., “*difficultly*” → “*difficulty*”): Whisper Medium corrects 32%, Whisper Small 29%, Parakeet 23%, Wav2Vec2 6%.
- **R:VERB:FORM** ($n = 686$, verb form; e.g., “I enjoy *to travel*” → “I enjoy *travelling*”): Whisper Medium corrects 24%, Whisper Small 22%, Parakeet 21%, Wav2Vec2 5%.

The common thread is that these errors are solvable from local context: a model that has learned “many” is followed by a plural noun will tend to produce the plural form regardless of what the speaker actually articulated. This is why both Whisper’s autoregressive decoder and Parakeet’s FastConformer encoder correct them at substantial rates. The difference between the two is one of degree, not of kind: Parakeet’s encoder, through self-attention and 64,000 hours of supervised training, has absorbed many of the same morphosyntactic regularities that Whisper’s decoder learns through autoregressive conditioning. On R:NOUN:NUM, the most frequent type, both Whisper Small and Parakeet correct exactly 31% of errors. The gap widens on R:VERB:SVA (31% vs. 22%) and R:MORPH (29% vs.

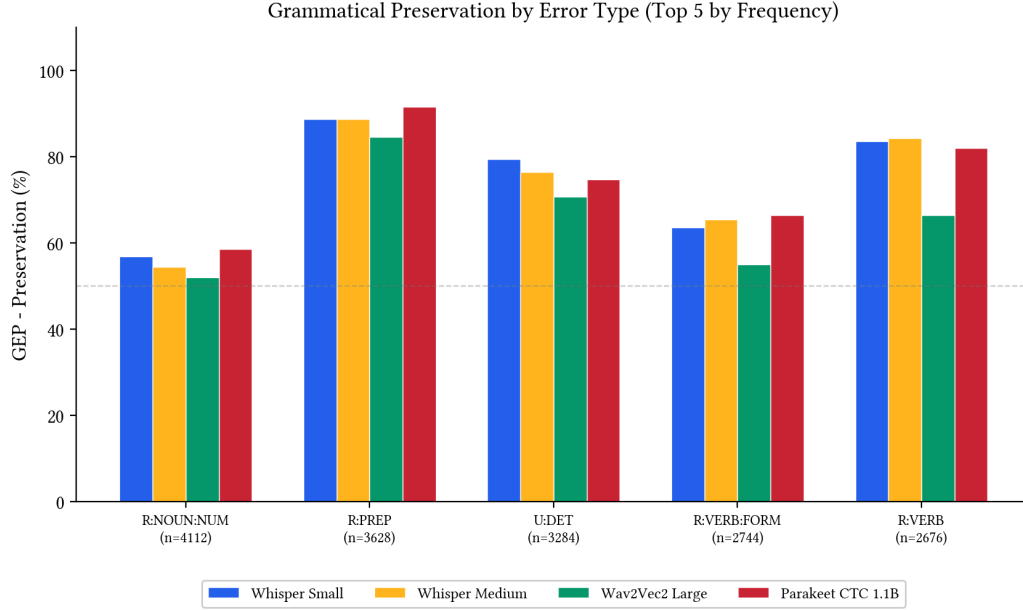


Figure 9: Grammatical preservation for the 5 most frequent error types (with n occurrences). Whisper models show consistently lower preservation rates in Replacement-type errors, indicating a greater tendency for auto-correction.

23%), where the autoregressive decoder’s explicit token-by-token conditioning provides a stronger corrective signal than the encoder’s implicit contextual representations.

Wav2Vec2, by contrast, shows correction rates below 6% across all morphosyntactic types. Its encoder, trained on only 960 hours via self-supervised pretraining, has not developed a strong enough implicit language model to systematically correct these patterns. For Wav2Vec2, non-preservation is overwhelmingly driven by mutation (transcription noise), not by intentional correction – consistent with the overall 90% mutation share among its non-preserved errors reported in Figure 6.

6.2.1.2. Lexical Errors

Not all error types are equally susceptible. Errors involving **open-class lexical choices** – replacing one verb, noun, or adjective with another – are almost never corrected by any model:

- **R:VERB** ($n = 669$, verb replacement; e.g., “make a decision” → “take a decision”): correction rates of 1–2% across all models.
- **R:NOUN** ($n = 465$, noun replacement; e.g., “effect” → “affect”): correction rates of 1–5%.
- **R:ADJ** ($n = 154$, adjective replacement; e.g., “big problem” → “large problem”): correction rates of 1–4%.

These errors involve semantic choices that cannot be resolved from local grammatical context alone. The learner used a valid word that happens to be the wrong one; predicting

the *intended* word would require understanding the broader communicative intent, which no current ASR model achieves. The low correction rates confirm that when these types are not preserved (particularly for Wav2Vec2, where R:NOUN:INFL reaches only 9% preservation), the cause is transcription noise rather than targeted auto-correction.

6.2.1.3. Missing-Word Errors

Missing-word errors (M : *xxx*), where the learner omits a required word, reveal the sharpest architectural contrast. An autoregressive decoder can *insert* a token that was never spoken by generating it as a high-probability continuation of the preceding context. A CTC model classifies existing audio frames and has a much more limited ability to produce words without any acoustic grounding. For tutoring applications, this difference is a **disadvantage** of Seq2Seq models: the decoder silently inserts the word the learner failed to produce, destroying diagnostic evidence that the learner omitted it.

- **M:DET** ($n = 618$, missing determiner; e.g., “I have _ *dog*” → “I have *a dog*”): Whisper Medium corrects 18%, Whisper Small 15%, Parakeet 11%, Wav2Vec2 5%. Missing determiners are only moderately auto-corrected by Seq2Seq models and rarely inserted by CTC models, indicating that models do not frequently hallucinate missing “the” or “a” tokens purely from linguistic context without acoustic grounding.
- **M:VERB:FORM** ($n = 140$, missing verb form; e.g., “I want _ *go*” → “I want *to go*”): Whisper Small and Medium correct 34–36%, but Parakeet corrects only 16%. This gap isolates the decoder’s generative advantage: inserting a function word like “to” between “want” and “go” requires generating a token with no acoustic evidence, which the decoder does naturally by predicting the most probable next token but which the CTC encoder cannot do as easily. In Figure 7, Wav2Vec2 preserves 98% of these errors (it almost never inserts the missing word), compared to 61–65% for Whisper.

The M:VERB:FORM contrast is particularly instructive because it highlights a case where CTC’s architectural limitation becomes an advantage for error preservation. Whisper’s decoder, following the learned probability $P(\text{to} \mid \text{I want})$, inserts the missing infinitive marker over a third of the time. Parakeet’s encoder, which must map each audio frame to a token, lacks a comparable mechanism for inserting unspoken words, and consequently preserves the omission more faithfully.

6.2.2. Correction by Grammatical Category

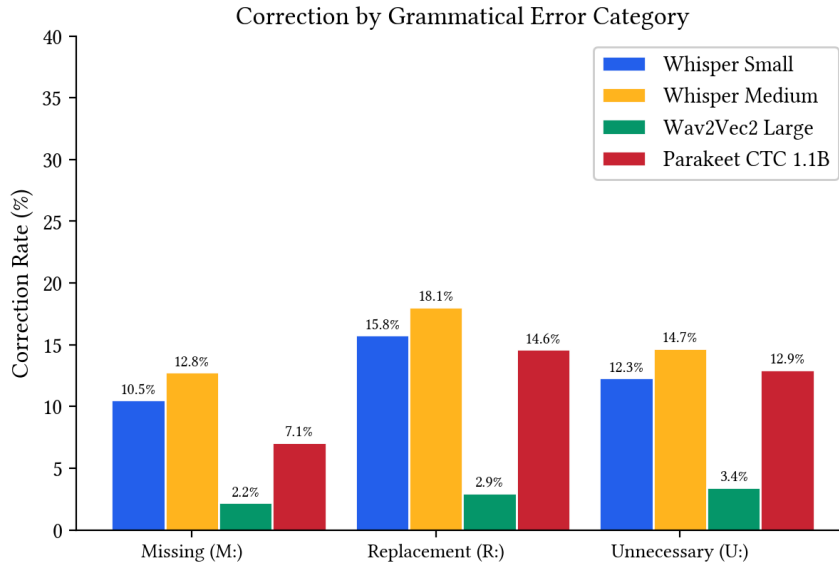


Figure 10: Correction rate by error category: Missing (M:), Replacement (R:), and Unnecessary (U:).

Whisper models correct significantly more errors in all categories.

Figure 10 aggregates the per-type correction rates into three broad ERRANT families, confirming that the patterns identified in the previous section are consistent across categories rather than driven by a few outlier types. Auto-correction affects all three categories, but at different magnitudes.

The most informative pattern is the **systematic gap between Whisper Medium and Whisper Small**. Across all three categories, Medium corrects more errors than Small: 12.8% vs. 10.5% on Missing, 18.1% vs. 15.8% on Replacement, and 14.7% vs. 12.3% on Unnecessary — a consistent 2 pp difference. Since both models share the same Transformer architecture and differ only in scale (769M vs. 244M parameters), this confirms that the additional decoder capacity translates directly into more aggressive auto-correction. For tutoring applications, the larger Seq2Seq model is systematically worse at preserving learner errors despite achieving nearly identical WER (18.23% vs. 18.39%).

Parakeet CTC 1.1B occupies an intermediate position: its correction rates on Replacement (14.6%) and Unnecessary (12.9%) errors fall between the two Whisper models, consistent with the implicit language model in its FastConformer encoder being effective at recognizing when a word should be different or absent. On Missing errors, however, Parakeet drops to 7.1% — below both Whispers — reflecting the limited ability of CTC to insert unspoken tokens, as discussed in the per-type analysis of M:VERB:FORM above.

Wav2Vec2 remains uniformly low across all categories (2.2–3.4%). Its highest correction rate is on Unnecessary errors (3.4%), reflecting its lack of an explicit language model to generate missing words or intentionally correct replacements.

6.3. WER–GEP Trade-off

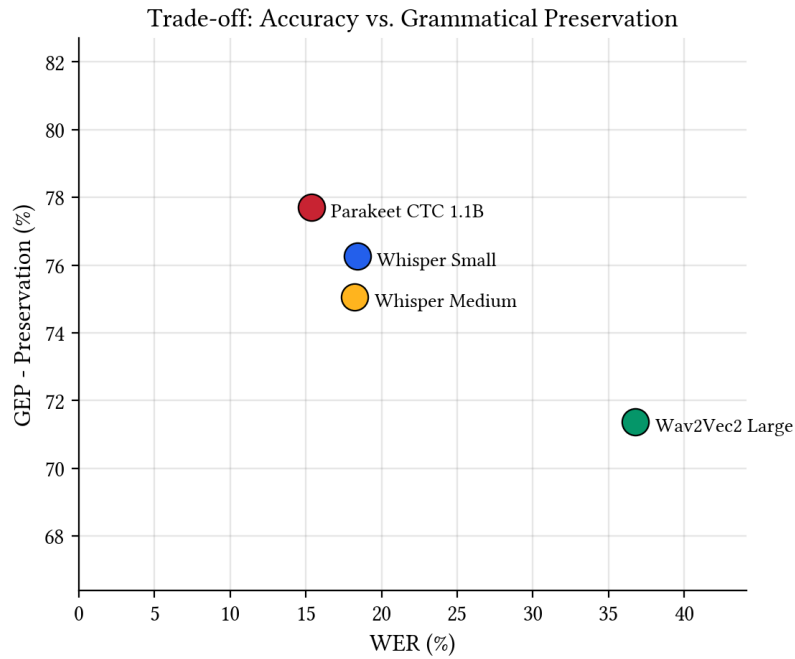


Figure 11: Trade-off between transcription accuracy (WER) and grammatical error preservation (GEP).

The ideal position is the top-left corner (low WER, high GEP).

Figure 11 synthesizes the fundamental trade-off: **Parakeet CTC 1.1B dominates the Pareto frontier**, with the best WER (15.35%) **and** the highest grammatical preservation (77.7%). Whisper Small offers the second best balance (WER 18.39%, GEP 76.3%). Whisper Medium, despite its greater capacity, **barely improves WER** (18.23%) but **increases auto-correction** (16.4%), placing it in a sub-optimal position. Wav2Vec2 occupies an extreme point with low practical utility (WER 36.76%, mutation 25.7%).

This result is particularly significant: Parakeet demonstrates that a CTC model with sufficient training data (64,000 hours) and a modern architecture (FastConformer) can outperform Seq2Seq models in accuracy **without** sacrificing grammatical error preservation.

6.4. Eval-Set Stability

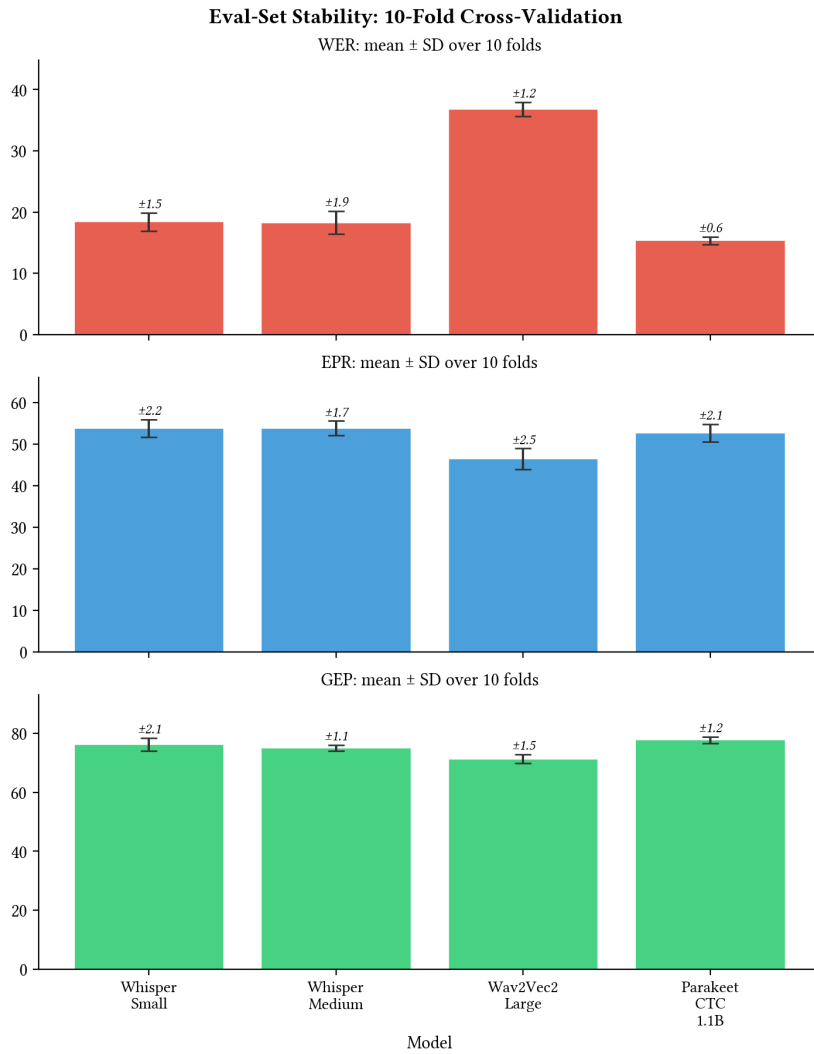


Figure 12: 10-fold cross-validation on the eval set (3,209 utterances). Bars show the mean metric value across folds; error bars indicate ± 1 standard deviation. Low variance confirms that no single subset of utterances disproportionately drives the aggregate results.

To verify that the reported metrics are not an artifact of a particular subset of utterances, a 10-fold cross-validation was performed on the eval set: the 3,209 utterances were randomly partitioned into 10 disjoint folds, and WER and GEP were recomputed independently on each fold. Figure 12 shows that both metrics exhibit low variance across folds for every model, with standard deviations below 2 percentage points in all cases. This confirms that the eval set is large and diverse enough for the aggregate metrics to be robust and that the results are not driven by a small number of outlier utterances.

6.5. Analysis by Speaker Proficiency Level

The S&I corpus includes continuous SLA (*Spoken Language Assessment*) scores for each exam session, on a scale from 2.0 to 6.0. These scores reflect the overall proficiency of

the speaker evaluated by human examiners. To study how ASR metrics vary according to the learner’s level, each utterance was associated with its session’s SLA score, and the 300 speakers in the eval set were grouped into three proficiency bands:

Band	SLA Range	Speakers	Utterances	GEP-scored Errors
Low	< 3.5	70	699	2,336
Medium	3.5 – 4.5	178	1,873	6,225
High	> 4.5	52	568	1,836

The three bands cover 3,140 of the 3,209 eval utterances; the remaining 69 utterances correspond to sessions for which an overall SLA score was not assigned in the corpus, and are therefore excluded from the proficiency stratification but retained in every other section of this work.

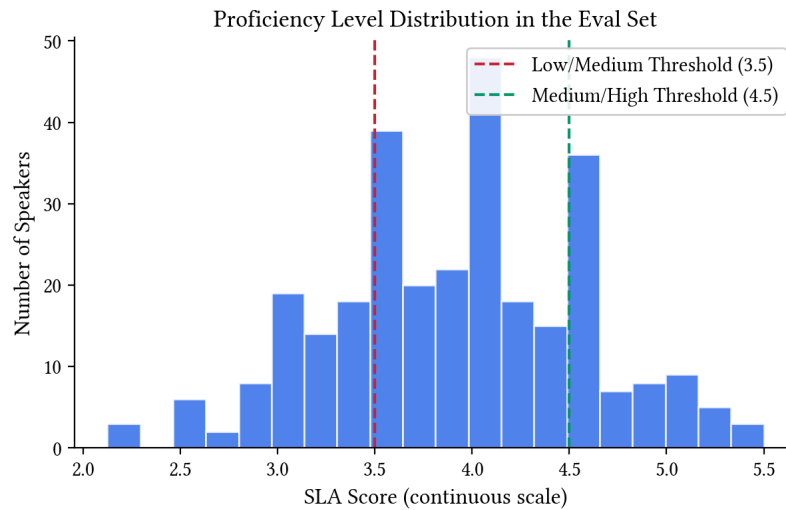


Figure 13: Distribution of SLA scores for the 300 speakers in the eval set. Dashed lines indicate the thresholds between proficiency bands.

Figure 13 shows that the score distribution is concentrated in the medium band (3.5–4.5), with a mean of 3.92 and a median of 3.88, reflecting a heterogeneous population of learners typical of assessment platforms like Speak & Improve.

6.5.1. WER by Proficiency Level

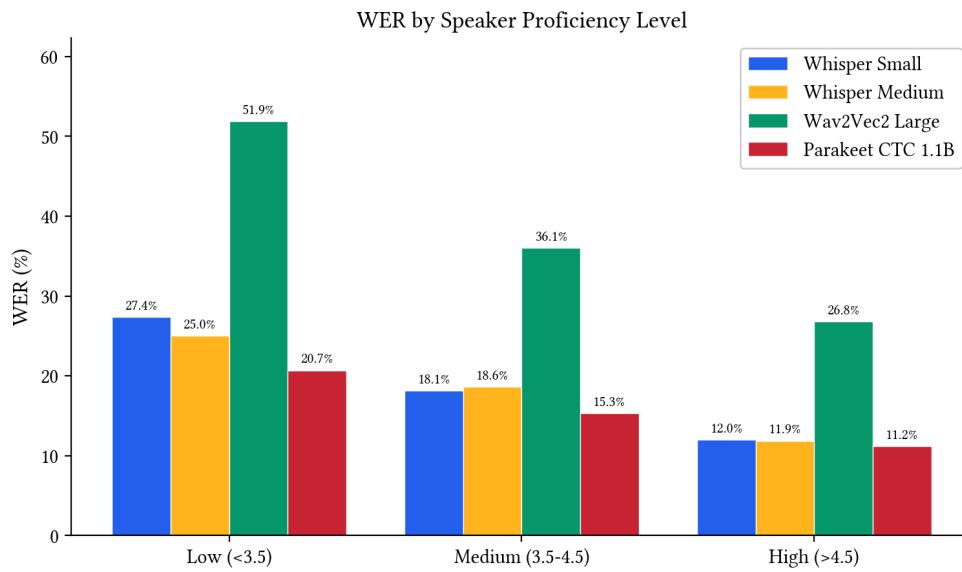


Figure 14: WER by model and speaker proficiency level. The vertical axis shows WER (%).

Figure 14 reveals a monotonic and expected relationship between proficiency level and WER: low-proficiency speakers generate a significantly higher WER across all models. Wav2Vec2 Large is the most affected, with a WER of 51.9% for low-proficiency speakers compared to 26.8% for high-proficiency (a degradation of 25%). Parakeet CTC 1.1B shows the lowest sensitivity to level: its WER goes from 20.7% (low) to 11.2% (high), a difference of only 9.5 points. Whisper Small and Medium show similar patterns to each other, with WERs in the 25–27% range for low level and 12% for high level.

6.5.2. GEP and Auto-Correction by Proficiency Level

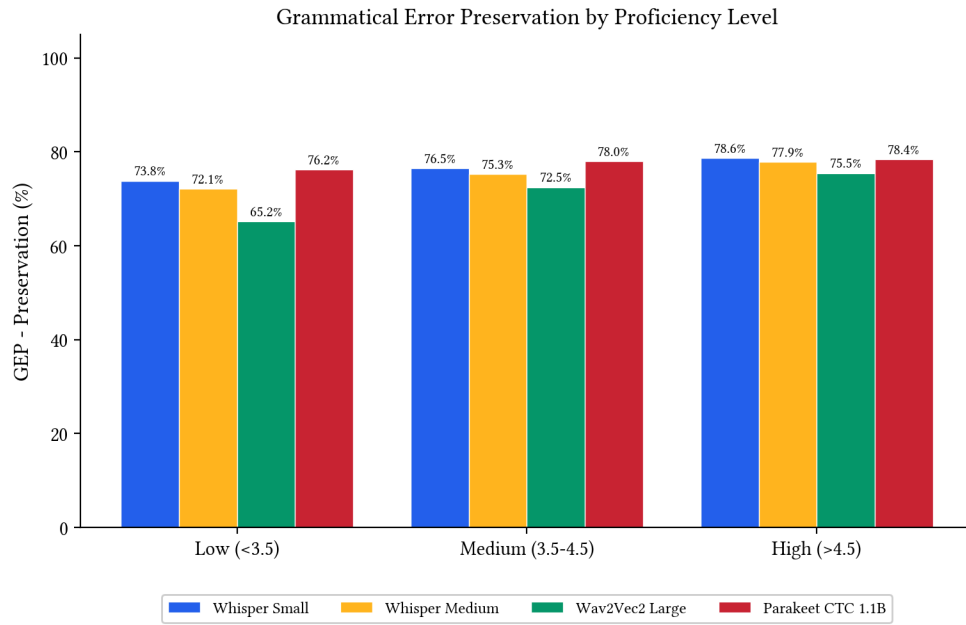


Figure 15: Grammatical error preservation (GEP) rate by proficiency level. High-proficiency speakers show greater preservation of their errors.

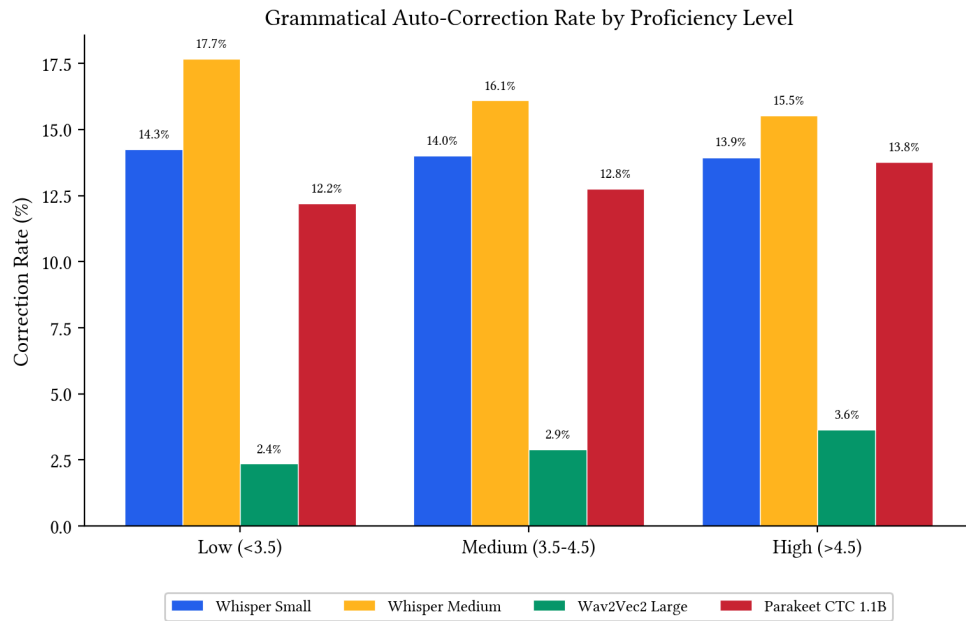


Figure 16: Grammatical auto-correction rate by proficiency level. Whisper models correct more errors at all levels, but the differences are modest compared to the mutation rate differences.

GEP results stratified by proficiency (Figure 15, Figure 16) reveal a consistent pattern across all models: **errors from low-proficiency speakers are preserved less**. The difference is most pronounced in Wav2Vec2 Large (65.2% for low vs. 75.5% for high, $\Delta = 10.3$ pp), followed by Whisper Small (73.8% vs. 78.6%, $\Delta = 4.8$ pp) and Whisper Medium (72.1% vs.

77.9%, $\Delta = 5.8$ pp). Parakeet CTC 1.1B shows the lowest variation (76.2% vs. 78.4%, $\Delta = 2.2$ pp), suggesting that its CTC architecture is more robust to the speaker’s proficiency level. Importantly, as with the overall correction rate analysis (Figure 6), the raw correction rate by proficiency must be interpreted alongside the mutation rate. The primary cause of lower preservation at low proficiency levels is not higher auto-correction by the model, but rather that lower-level speech presents more acoustic difficulties (atypical pronunciation, frequent disfluencies) that increase the mutation rate. This has a pedagogically critical implication: errors from speakers who most need corrective feedback are the ones ASR models preserve worst.

6.5.3. WER–GEP Trade-off by Proficiency Level

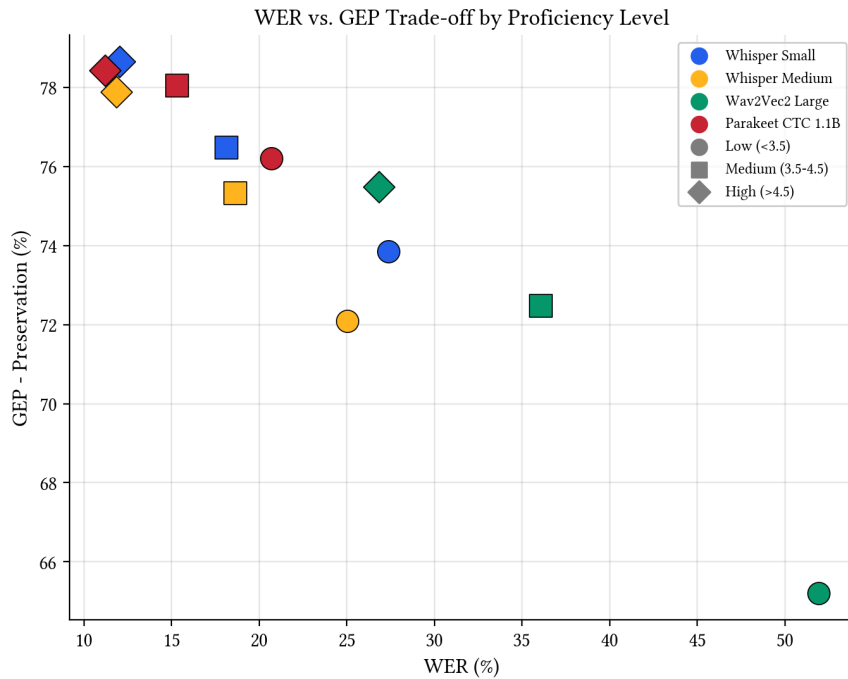


Figure 17: WER vs. GEP trade-off stratified by proficiency level. The ideal position is the top-left corner (low WER, high GEP). Each model generates three points (one per band), with markers differentiated by level.

Figure 17 visualizes the WER-GEP trade-off segmented by level. **Parakeet CTC 1.1B dominates the Pareto frontier across all proficiency bands**, confirming its superiority in both accuracy and grammatical preservation regardless of the speaker’s level. The point dispersion is highest for Wav2Vec2, which exhibits the worst degradation for low-proficiency speakers.

7. Lightweight Models: Viability in Resource-Constrained Environments

The previous results identify Parakeet CTC 1.1B (1,063M parameters) as the optimal model. However, its size presents concrete deployment challenges: at FP16 precision the model weights alone occupy approximately 2.1 GB of memory, and peak VRAM during inference reaches approximately 6.7 GB in offline mode. These requirements exceed the capabilities of most mobile devices, which typically offer 4–6 GB of shared memory for all running applications, and are impractical for client-side web inference where WebGPU memory budgets are even more constrained. This section investigates whether significantly smaller models (61–182M parameters), whose weights fit in 120–360 MB at FP16, can offer an acceptable trade-off between efficiency and grammatical error preservation.

7.1. Selected Models

Three lightweight models covering the spectrum of scaled-down architectures are evaluated:

Model	Parameters	Architecture	Type	Memory (FP16)
Parakeet TDT-CTC 110M	114M	FastConformer + TDT/CTC	Hybrid CTC	230 MB
Canary 180M Flash	182M	FastConformer + Transformer dec.	Encoder-Decoder	360 MB
Moonshine Base	61M	Encoder-Decoder	Encoder-Decoder	120 MB

Parakeet TDT-CTC 110M uses a FastConformer encoder with hybrid TDT (Token-and-Duration Transducer) and CTC decoding. With 114M parameters, it is the compact version of the evaluated Parakeet 1.1B, trained on 36,000 hours of English speech. As a CTC-based model, it lacks an autoregressive decoder, making it a key test of whether the CTC architectural advantage scales down efficiently.

Canary 180M Flash (Puvvada et al., 2024) combines a 17-layer FastConformer encoder with a 4-layer Transformer decoder. With 182M parameters, it is a multilingual model (English, German, French, Spanish) trained on approximately 85,000 hours of speech. Its 4-layer decoder, although small, introduces an autoregressive component that could favor grammatical auto-correction.

Moonshine Base (Jeffries et al., 2024) employs an encoder-decoder architecture optimized for real-time transcription on low-power devices. With only 61M parameters, it is the

smallest model in this comparison, trained on approximately 200,000 hours of speech. Its ultra-reduced capacity makes it an extreme case of efficiency, with the expectation of significant degradation in accuracy but potentially low auto-correction.

7.2. Comparative Results

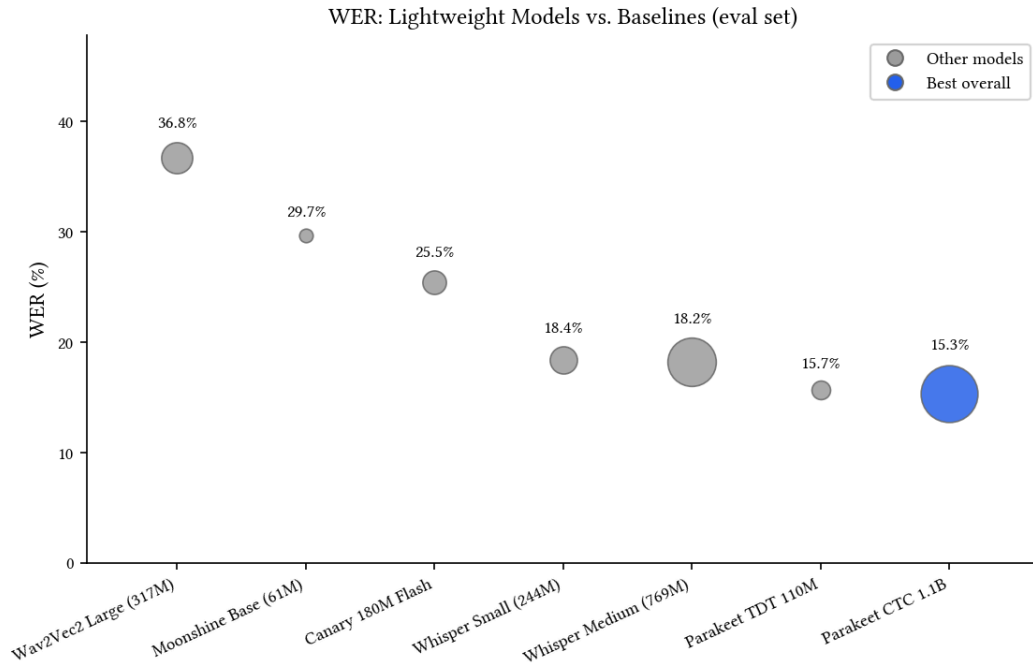


Figure 18: WER comparison among all models, ordered from worst to best. Circle size is proportional to model parameter count. The best model is highlighted in blue.

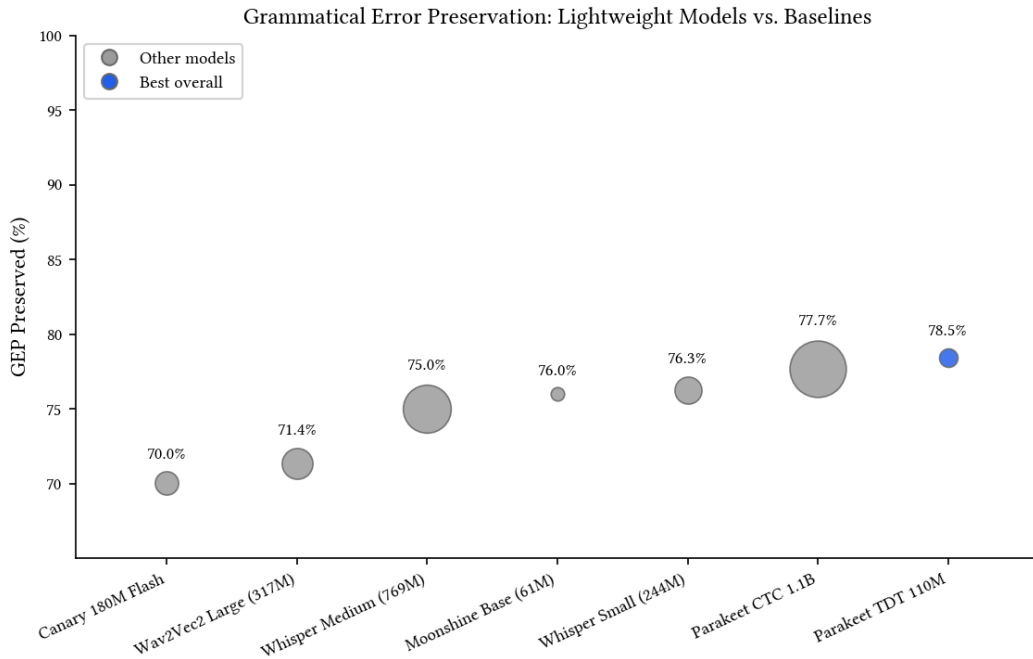


Figure 19: Grammatical error preservation (GEP) comparison among all models, ordered from worst to best. Circle size is proportional to model parameter count.

Model	Params	WER (%)	GEP (%)	Corr. (%)	Mut. (%)
Parakeet CTC 1.1B	1,063M	15.35	77.7	12.8	9.5
Parakeet TDT-CTC 110M	114M	15.67	78.5	12.2	9.3
Whisper Small	244M	18.39	76.3	14.0	9.7
Whisper Medium	769M	18.23	75.0	16.4	8.6
Canary 180M Flash	182M	25.47	70.0	13.4	16.5
Moonshine Base	61M	29.70	76.0	9.2	14.8
Wav2Vec2 Large	317M	36.76	71.4	2.9	25.7

7.2.1. Parakeet TDT-CTC 110M: Exceptional Performance at Reduced Scale

The most outstanding result of this section is that **Parakeet TDT-CTC 110M reproduces virtually the performance of its 10x larger version**. With only 114M parameters (10.7% of the 1.1B’s size), it achieves:

- **WER 15.67%** vs. 15.35% for the 1.1B — a difference of just 0.32 percentage points.
- **GEP 78.5%** vs. 77.7% for the 1.1B — slightly **higher** than the large model.
- **Correction rate 12.2%** vs. 12.8% — marginally lower, suggesting a slightly reduced tendency to auto-correct at this scale.

This result is remarkable for multiple reasons: (1) it demonstrates that the FastConformer architecture with CTC/TDT decoding scales exceptionally efficiently for L2 speech; (2) it suggests that 36,000 hours of training data are sufficient to reach accuracy comparable to 64,000 hours; and (3) it confirms that a 114M parameter model deployable on mobile devices can offer the same level of grammatical preservation as >1,000M parameter models.

7.2.2. Canary 180M Flash: Decoder Impact at Reduced Scale

Canary 180M Flash presents an interesting case: despite having more parameters than Parakeet 110M (182M vs. 114M), its performance is significantly inferior:

- **WER 25.47%** — 10 points worse than Parakeet 110M and 7 points worse than Whisper models.
- **GEP 70.0%** — the second-worst preservation, only above Wav2Vec2 Large.
- **Correction rate 13.4%** — comparable to Parakeet 1.1B (12.8%) and below both Whisper models.

The high deletion rate suggests that Canary produces systematically shorter transcriptions, as shown in the WER error breakdown below:

Model	Substitutions	Deletions	Insertions
Parakeet CTC 1.1B	7,175	5,181	7,843
Parakeet TDT-CTC 110M	7,697	5,200	7,730
Whisper Small	8,210	5,526	10,463
Whisper Medium	7,432	5,718	10,849
Canary 180M Flash	7,139	19,212	7,165
Moonshine Base	12,821	7,482	18,792
Wav2Vec2 Large	29,981	3,263	15,142

Canary’s 19,212 deletions are nearly four times those of Parakeet 110M (5,200), confirming that the model systematically truncates its output. The high WER is partially explained by its multilingual nature: the model must distribute its limited capacity (182M) across four languages and translation tasks, resulting in inferior performance compared to monolingual English models of similar size. Furthermore, its 4-layer decoder, although small, introduces a tendency towards auto-correction (13.4%) **higher** than that of pure CTC models, confirming that even a minimal decoder induces corrective behavior.

7.2.3. Moonshine Base: Limits of Ultra-Lightweight Models

Moonshine Base (61M parameters) represents the lower end of the spectrum:

- **WER 29.70%** — significantly worse than Parakeet 110M, but better than Wav2Vec2 Large.
- **GEP 76.0%** — competitive with Whisper Medium (75.0%).
- **Correction rate 9.2%** — the lowest among encoder-decoder models in this comparison.

The most interesting finding is the combination of a low correction rate (9.2%) with relatively high GEP preservation (76.0%). Moonshine, having only 61M parameters, lacks the linguistic capacity necessary to “correct” grammatical errors which means it preserves them by default. Its main weakness is the mutation rate (14.8%), reflecting its difficulty in correctly transcribing L2 learner speech.

7.3. Efficiency–Performance Trade-off

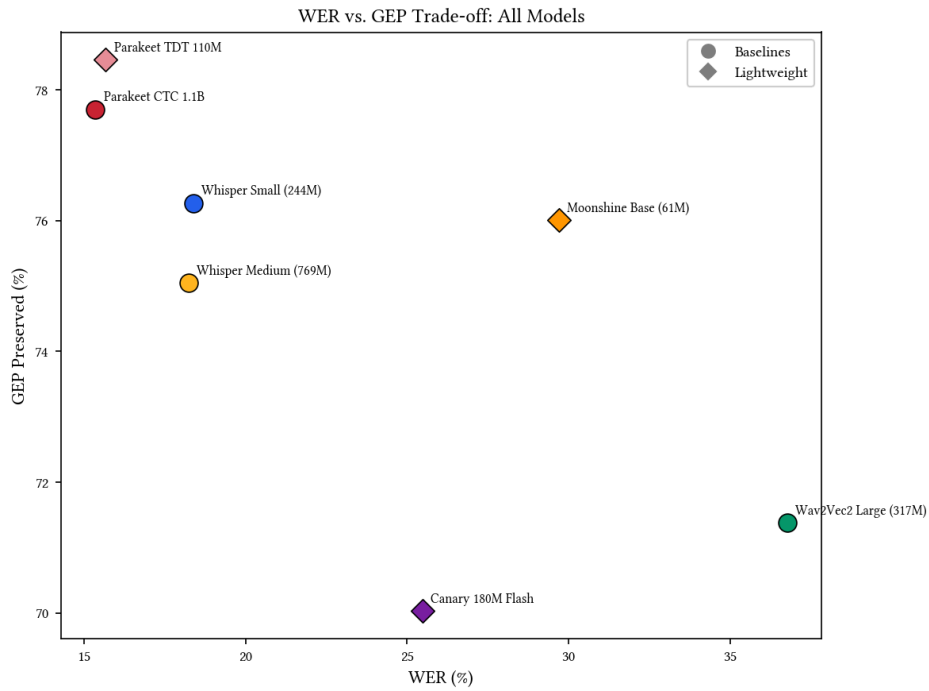


Figure 20: WER vs. GEP trade-off including lightweight models. Diamonds represent lightweight models, circles baselines. The ideal position is the top-left corner (low WER, high GEP).

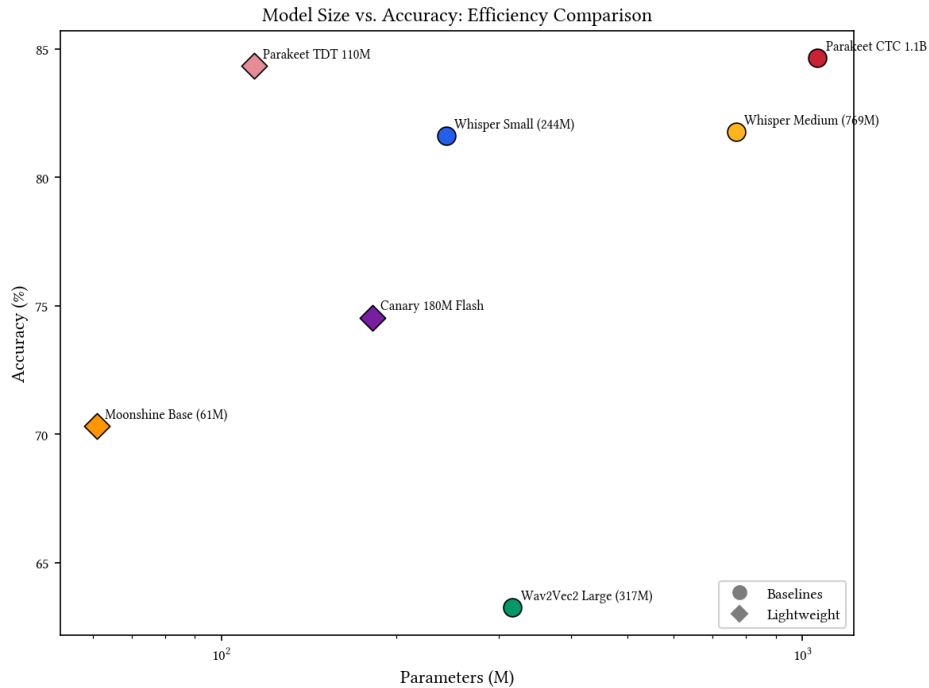


Figure 21: Model size (parameters) vs. transcription accuracy (100% – WER). Circles represent baselines, diamonds lightweight models. The logarithmic scale of the X-axis reveals the exceptional efficiency of Parakeet TDT-CTC 110M. The ideal position is the top-left corner (fewer parameters, higher accuracy).

Figure 20 shows that **Parakeet TDT-CTC 110M** lies on the **Pareto frontier** alongside its 1.1B version, dominating all other models on both axes. Figure 21 reveals the exceptional

efficiency of the Parakeet family: with 10x fewer parameters, the 110M model achieves performance virtually identical to the 1.1B model.

7.4. Parakeet 110M vs. 1.1B: Comparison by Proficiency Level

To delve deeper into the equivalence between Parakeet TDT-CTC 110M and its 1.1B version, results were stratified by speaker proficiency level using the same SLA bands defined earlier (Low < 3.5, Medium 3.5–4.5, High > 4.5).

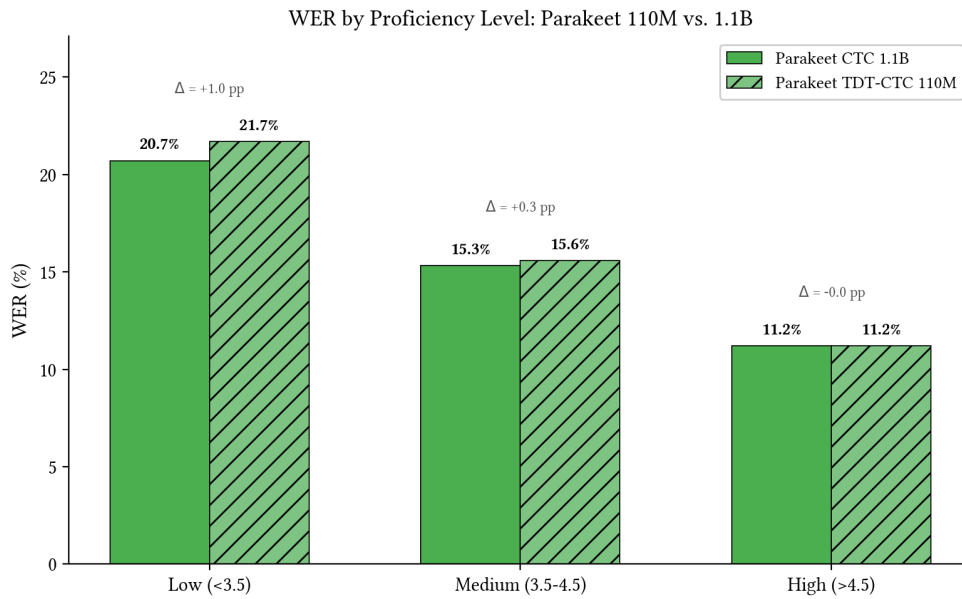


Figure 22: WER by proficiency level for Parakeet CTC 1.1B and Parakeet TDT-CTC 110M. Differences are minimal across all levels (Δ indicates the difference 110M – 1.1B in percentage points).

Model	Level	WER (%)	GEP (%)	Corr. (%)	Mut. (%)	Errors
Parakeet CTC 1.1B	Low	20.71	76.2	12.2	11.6	2,336
Parakeet TDT-CTC 110M	Low	21.69	77.1	11.1	11.7	2,336
Parakeet CTC 1.1B	Medium	15.33	78.0	12.8	9.2	6,225
Parakeet TDT-CTC 110M	Medium	15.58	78.8	12.3	8.9	6,225
Parakeet CTC 1.1B	High	11.21	78.4	13.8	7.8	1,836
Parakeet TDT-CTC 110M	High	11.21	79.1	13.2	7.7	1,836

Figure 22 reveals that the WER difference between both models is minimal across all proficiency levels. For low-proficiency speakers, Parakeet 110M presents a WER of 21.69% compared to 20.71% for the 1.1B, a difference of just 1.0 percentage point. In the medium level, the difference shrinks to 0.3 pp (15.58% vs. 15.33%). In the high level, both models achieve **exactly the same WER** (11.21%), demonstrating that the additional capacity of the 1.1B model is irrelevant when the speech is of high quality.

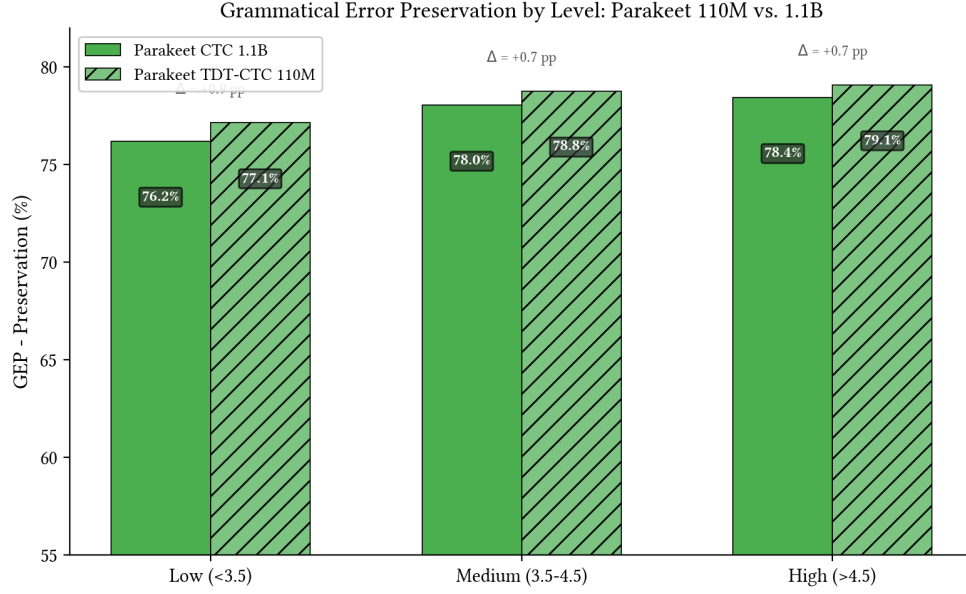


Figure 23: Grammatical error preservation (GEP) by proficiency level. Parakeet 110M matches or slightly exceeds the 1.1B across all bands.

The most notable result is observed in grammatical preservation (Figure 23): **Parakeet TDT-CTC 110M matches or slightly outperforms the 1.1B model across all three proficiency bands.** At the low level, the 110M’s GEP is 77.1% versus 76.2% for the 1.1B ($\Delta = +0.9$ pp); at the medium level, 78.8% vs. 78.0% ($\Delta = +0.8$ pp); and at the high level, 79.1% vs. 78.4% ($\Delta = +0.7$ pp). This consistent advantage of the compact model, though small, suggests that its fewer parameters confer a marginally lower capacity to “correct” grammatical errors, which paradoxically proves beneficial for this application.

This interpretation is corroborated by the auto-correction rate: the 110M systematically corrects fewer errors than the 1.1B at all levels (11.1% vs. 12.2% in low, 12.3% vs. 12.8% in medium, and 13.2% vs. 13.8% in high). The difference is most pronounced in low-proficiency speakers ($\Delta = -1.1$ pp on correction rate), where errors are more frequent and detectable by the implicit language model.

Additionally, both models show the same monotonic trend of degradation with proficiency level: GEP goes from 76–77% at the low level to 78–79% at the high level, with a total variation of only 2.0–2.2 pp. This stability across levels is an inherent property of the CTC architecture with FastConformer, which is faithfully preserved when reducing the model from 1,063M to 114M parameters.

7.5. Deployment Implications

These results have direct implications for deploying CALL systems in resource-constrained environments:

1. **Parakeet TDT-CTC 110M is viable for production on mobile devices:** With only 114M parameters (230 MB at FP16) and inference speeds over 300 times faster than real-time, it offers top-tier performance with minimal computational requirements.
2. **Multilingual models penalize monolingual performance at small scales:** Canary 180M Flash, despite having more parameters, performs worse than Parakeet 110M on English, suggesting that multilingual capacity comes at a cost at small scales, though further research is needed to isolate this effect from other architectural differences.
3. **Ultra-lightweight models (<100M) are not viable for tutoring:** Moonshine Base, with a WER of 29.7%, introduces too much noise into the transcription for reliable pedagogical use, although its low auto-correction (9.2%) suggests that with accuracy improvements it could become interesting.
4. **The CTC/TDT architecture maintains its advantage at any scale:** Even at 114M parameters, CTC models show a lower tendency to auto-correct than models with a decoder, confirming that architectural choice —not size— is a much more important factor when it comes to corrective behavior.

8. Experimentation: Fine-Tuning for Error Preservation

The comparative evaluation in the previous chapter establishes that error preservation is largely a property of the model architecture: CTC encoders behave differently from autoregressive decoders, and the Whisper family in particular auto-corrects grammatical errors at rates between 10 and 18 percent. A natural follow-up question is whether a Whisper-class model can be adapted, by *fine-tuning* on L2 speech, to preserve grammatical errors without sacrificing its overall transcription accuracy. This chapter explores that question with a single, targeted experiment: a Direct Preference Optimization (DPO) pass on Whisper Small, trained on a small curated set of preference pairs.

The experiment is intentionally narrow. Rather than treating fine-tuning as a generic domain adaptation problem, the recipe below is designed around two specific convictions about what could go wrong, and what each design choice is supposed to prevent. The result on the full *eval* partition (3,209 utterances, 10,397 ERRANT-scored grammatical edits) is a model that improves both WER (−1.12 percentage points) and GEP (+4.0 percentage points) simultaneously, moving Whisper Small up and to the left of its baseline on the trade-off plot.

8.1. Hypothesis and Design Rationale

Two design decisions shape the entire experiment. The first concerns **what data the model sees**; the second concerns **which loss function it minimizes**. Both follow from the same observation: when the objective is as narrow as “preserve a specific grammatical error that the baseline auto-corrects”, training signal should also be narrow, both in coverage and in shape.

Selectivity over volume in the training data. Whisper Small has been pre-trained on roughly 680,000 hours of audio. Fine-tuning it on the 3,883 train-partition utterances of the S&I corpus, an amount that is several orders of magnitude smaller, is not realistic as a generic adaptation: there is no plausible parameter trajectory in which a corpus that small teaches a 244-million-parameter model “how to transcribe L2 speech”. The phenomenon we want to change is much more local than that, only the moments when the baseline silently restores a grammatical error to its standard form. The corpus contains thousands of utterances where Whisper transcribes faithfully and there is nothing to learn; including them in training would mainly contribute regularization noise. The training set was therefore filtered down to only those utterances where the baseline Whisper Small hypothesis differs from the reference precisely by auto-correcting a whitelisted ERRANT edit, leaving 1,728 candidates of which 1,274 survived the quality filters described below.

A loss function that does not conflict with Whisper’s output style. The most obvious training objective, a cross-entropy loss between the reference transcription and Whisper’s autoregressive output, is structurally hostile to the goal of this work. The S&I reference transcripts are normalized text from human annotators (lowercase, structural disfluency markers such as (%hesitation%), partial-word markers such as (univers-), no punctuation). Whisper, on the other hand, emits cased English with punctuation and renders disfluencies as natural fillers (“um”, “uh”). A cross-entropy loss on those references penalizes the model on every position where the two conventions diverge, which is essentially every word, even when the model is otherwise transcribing perfectly. The literature on token-level masking and word-to-subword alignment offers approximate work-arounds, but every alignment-based method introduces brittle bookkeeping (one mis-alignment per pair will systematically nudge gradients in the wrong direction), and crucially the loss continues to require Whisper-incompatible reference text at the position of every supervised token.

Direct Preference Optimization (DPO, Rafailov et al., 2023) eliminates the alignment problem altogether by operating on full strings the model could plausibly emit, expressing supervision as a **preference** between two candidate transcriptions of the same audio rather than as an absolute target. If both candidates are written in Whisper’s native style and they differ only at the grammatical error site, the gradient signal is restricted to that one site by construction, with no masking machinery needed.

8.2. Building the DPO Preference Pairs

DPO data construction and training pipeline

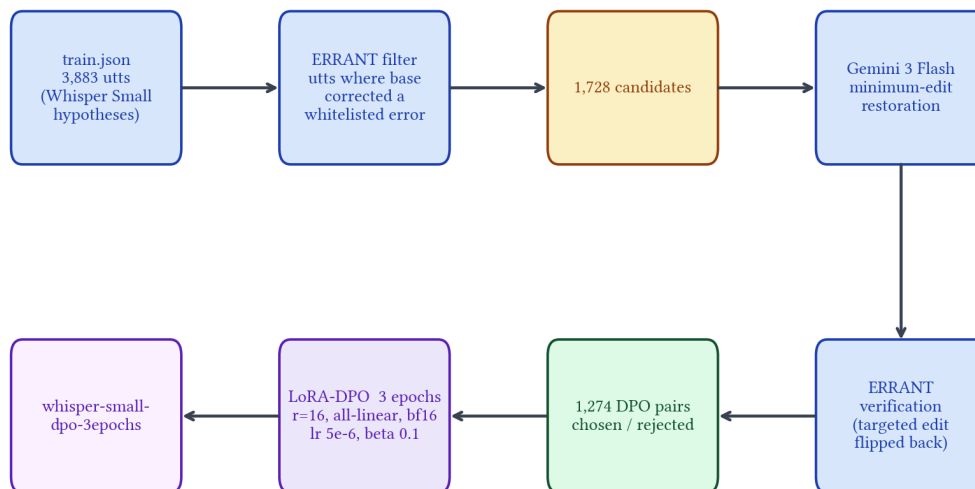


Figure 24: Construction of the DPO preference pairs and the training loop. From the 3,883 utterances of the S&I train partition, only those where the Whisper Small baseline auto-corrected a whitelisted ERRANT edit are kept (1,728 candidates). For each such utterance, Gemini 3 Flash makes the minimum edit to the Whisper hypothesis that restores the targeted error, and ERRANT verifies the result. The 1,274 pairs that survive the verification step are used to train Whisper Small for 3 epochs with LoRA-DPO.

The pair construction pipeline is summarized in Figure 24. Three steps move from the raw train partition to the final preference set; each step exists to enforce a specific property of the pairs.

Step 1 — Candidate selection (`build_finetune_pool.py`). The 3,883 train-partition utterances are transcribed with the base Whisper Small model and run through ERRANT against the reference. Only utterances where ERRANT reports at least one auto-correction belonging to a whitelisted error type are kept (the same set of 39 ERRANT categories that are scored in the evaluation chapter). This produces 1,728 candidate utterances. Selectivity is the point: every training example is by construction an instance of the phenomenon we want to change.

Step 2 — Minimum-edit error restoration (`restore_errors_llm.py`). For each candidate, the goal is to construct a Whisper-style transcript that **preserves** the learner’s error, to be used as the chosen sequence. Writing such a transcript programmatically is harder than it sounds: it must keep Whisper’s casing, punctuation and disfluency rendering exactly as the baseline produced them, and modify **only** the tokens that correspond to the targeted ERRANT edits. Naive string-replacement on the Whisper hypothesis fails because the error tokens are not always literal substrings of the hypothesis (Whisper may have

transcribed surrounding words slightly differently, may have inserted or merged words, or may have used a different morphological form than the one in the reference).

A short prompt asks Gemini 3 Flash to perform exactly this restoration: it receives the reference transcript (with errors), the baseline Whisper hypothesis (with errors auto-corrected), and the specific ERRANT edits that were corrected, and is instructed to return the Whisper hypothesis modified by the minimum set of edits that restore those errors. The chosen sequence is the LLM’s output. The rejected sequence is the unmodified Whisper hypothesis.

Step 3 — ERRANT verification. The LLM output is run through ERRANT against the reference once more. A pair is accepted only if every targeted ERRANT edit that was originally corrected in the baseline is now annotated as preserved in the chosen sequence, and no new spurious edits have been introduced. This is the key quality gate: it guarantees that the chosen and rejected sequences differ at exactly the intended sites, and that the difference is the right one. Of the 1,728 candidates, 1,274 pairs (74%) pass verification and become the training set; the remaining 454 are dropped. The validation split (161 pairs from a held-out subset of the train partition) is constructed identically.

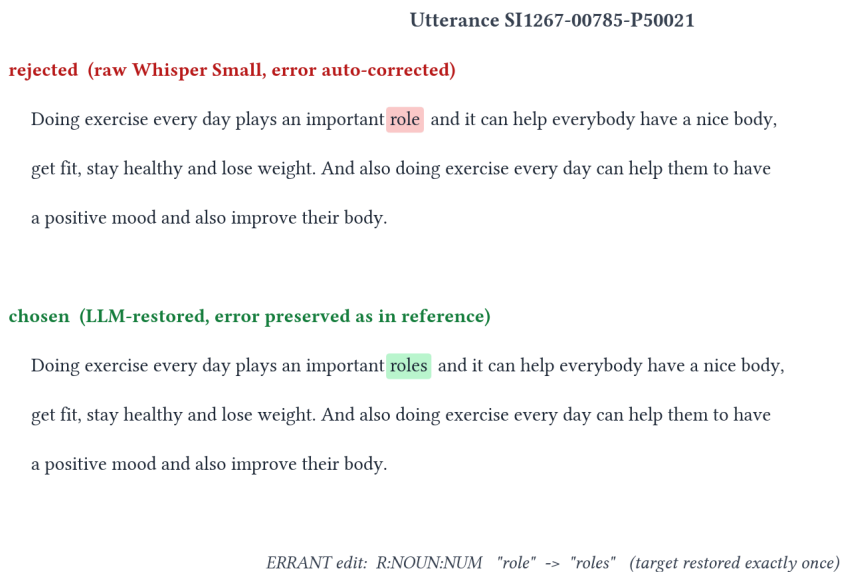


Figure 25: An example preference pair. The Whisper hypothesis already auto-corrected the learner’s number-agreement error (“important roles” → “important role”); ERRANT flagged this as an R:NOUN:NUM correction. The chosen sequence restores the original form, the rejected sequence is the Whisper hypothesis as emitted. The two strings differ at exactly one token.

Figure 25 shows a typical pair after this pipeline. Aside from the four characters at the highlighted position, the two sequences are byte-identical: same capitalization, same comma after “weight”, same “And also” rendering of the discourse marker. When the DPO

gradient is computed from these two sequences it can only push the policy’s probability mass between two near-identical strings, and the directional signal lives entirely at the error site.

8.3. Loss Function and Training Configuration

The DPO loss on a pair (y_w, y_l) for a given input x is

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\log \sigma \left(\beta \cdot \left(\log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right)$$

where π_θ is the policy (the LoRA-adapted Whisper), π_{ref} is a frozen reference policy (the same Whisper checkpoint with the LoRA adapters disabled), $\beta = 0.1$ controls how strongly preferences are enforced, and σ is the sigmoid. The objective rewards the policy for assigning higher relative probability to y_w (chosen, error preserved) than to y_l (rejected, error corrected), measured against the frozen reference. Because π_{ref} is a copy of the same network, “no change in behavior” corresponds to a loss of $-\log \sigma(0) \approx 0.69$; any deviation from the reference is implicitly regularised.

Why LoRA rather than full fine-tuning. Adaptation is implemented through Low-Rank Adaptation (LoRA, Hu et al., 2022), a parameter-efficient fine-tuning method that freezes the original weight matrix $W \in \mathbb{R}^{d \times k}$ of a target linear layer and learns a low-rank update $\Delta W = BA$ with $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$ and $\text{rank } r \ll \min(d, k)$. The base weights are never modified during training; only the much smaller A and B matrices receive gradients. At inference time the update is either kept as an additive adapter or folded back into W as $W' = W + BA$, in which case the runtime cost is identical to that of the unadapted model and no architectural change is needed in the decoding pipeline. Three properties of this scheme matter for the present setup:

- **Training cost and footprint.** With $r = 16$ on all linear layers of Whisper Small, the trainable parameter count drops from 244 M to 6.5 M (a $\approx 37 \times$ reduction), bf16 activations and the AdamW optimizer state fit on a single 12 GB consumer GPU, and the entire 3-epoch run completes in roughly 40 minutes. A full fine-tune on the same hardware would require aggressive gradient checkpointing and offloading and would still be impractical on a budget GPU.
- **Implicit regularization against drift.** Because the rank of every weight update is bounded above by r , the LoRA bottleneck physically restricts how far the policy can move from the baseline at any given layer. This is exactly the property the recipe wants: when the preference gradient at the error site is unambiguous, the adapter has enough degrees of freedom to follow it; when the signal is weak or noisy, the rank constraint forces the

model to fall back on its baseline behavior rather than overfit. This pairs naturally with the implicit reference-policy regularization of the DPO loss.

- **Cheap reference policy.** The reference π_{ref} in the DPO objective is just the same model with the LoRA adapters disabled, made possible by PEFT’s `enable_adapter_layers()` / `disable_adapter_layers()` mechanism. Each training step therefore performs four decoder forward passes (chosen and rejected, each under policy and reference) but only two encoder forward passes (the encoder output is shared across all four), and no second copy of the weights is loaded. With full fine-tuning a separate reference checkpoint would have to be held in memory, which is not feasible on a 12 GB card.

Setting	Value
Base model	openai/whisper-small.en (244 M params)
LoRA target modules	all linear layers (q_proj, k_proj, v_proj, out_proj, fc1, fc2) of both encoder and decoder
LoRA hyperparameters	rank 16, $\alpha = 32$, dropout 0.05, 6.5 M trainable params (2.6%)
DPO β	0.1
Optimiser	AdamW, weight decay 0.01, gradient clipping at 1.0
Learning rate	5×10^{-6} , linear warmup of 10% + cosine decay
Effective batch size	16 (physical batch $2 \times$ grad-accum 8)
Precision	bf16 mixed precision
Epochs	3
Hardware / wall time	single RTX 3060 (12 GB), $\approx$ 40 minutes total

The choice to apply LoRA to **all** linear layers, rather than the much more common q_proj/v_proj only of the original LoRA paper, was primarily a performance trade-off informed by more recent literature. Expanding the target set to all six linear modules (q_proj, k_proj, v_proj, out_proj, fc1, fc2) of both encoder and decoder roughly triples the adapter parameter count at the same rank, but in absolute terms this still amounts to only $\approx$ 6.5 M trainable parameters, 2.6% of the base model, and the impact on training throughput and GPU memory is marginal on this scale. In return, a growing body of work — including the QLoRA paper (Dettmers et al., 2023), which explicitly recommends targeting all linear layers, and analyses such as Raschka (2023) on practical LoRA recipes — reports that broader target coverage tends to yield equal or better downstream quality with no observable downside. Concretely, restricting LoRA to the attention projections only would leave the feed-forward sublayers (fc1, fc2) frozen, which together hold the majority of each transformer block’s parameters and most of its representational capacity, biasing the adapter to express any learned change as a shift in attention weights even when a feed-

forward change would be more natural. Targeting all linear modules removes that arbitrary asymmetry while remaining squarely in the parameter-efficient regime: the 6.5 M LoRA parameters can be folded into the base weights for inference, leaving runtime cost identical to the unadapted model.

8.4. Training Behavior



Figure 26: Training curves over 3 epochs on a held-out 100-utterance subset of the train partition. **Left:** validation GEP rises monotonically from 57.6% to 70.7%, while validation WER drifts upward from 3.4% to 10.7% against the chosen text. Because the chosen sequence is the LLM-restored Whisper-style output, baseline WER is artificially low; the upward drift reflects controlled stylistic adaptation toward the error-preserving sequence, not transcription decay. The full-eval result in Section 8.5 confirms that WER on the held-out partition actually improves. **Right:** training DPO loss falls and reward margin grows monotonically, indicating that the policy consistently learns to prefer the chosen sequence over the rejected one.

Figure 26 summarizes the training dynamics. The DPO loss falls from 0.69 (the initialization value, where the policy and reference are identical) toward a steady minimum, and the reward margin — the difference between chosen and rejected implicit rewards — grows monotonically over the three epochs, indicating that the policy is consistently learning to prefer the error-preserving sequence. Per-token accuracy (the fraction of validation pairs for which the model assigns higher likelihood to chosen than to rejected) climbs from 65% after epoch 1 to 94% after epoch 3.

The validation WER curve in the left panel needs careful reading. The reference used for this in-loop WER is the chosen string (the LLM-restored Whisper-style transcript with errors put back), not the human transcript. Because the baseline Whisper already emits a near-identical string at the chosen positions, the epoch 0 WER starts at an artificially low 3.4%. As the LoRA adapters learn to flip error sites toward the chosen form, they also introduce a small amount of style drift away from the baseline at other positions, so the WER-vs-chosen number ticks up. This is harmless: the full-eval results reported

in Section 8.5 show that WER measured against the actual human references *drops* from 18.39% to 17.27%. Validation GEP, in contrast, is computed against the reference annotations and rises monotonically, confirming that the preserved-error signal is reaching the intended behavior.

Validation in this run is computed by the same HuggingFace pipeline (chunked long-form decoding, `return_timestamps=True`) that is used at inference time, so the curves reflect the actual deployment path rather than an idealised greedy decode.

8.5. Results on the Full Eval Partition

The final checkpoint after epoch 3 was evaluated on the full S&I eval partition (3,209 utterances, 10,397 ERRANT-scored grammatical edits), using the same chunked-pipeline decoding path that produced the baseline numbers in the previous chapter. Figure 27 summarizes the headline metrics.

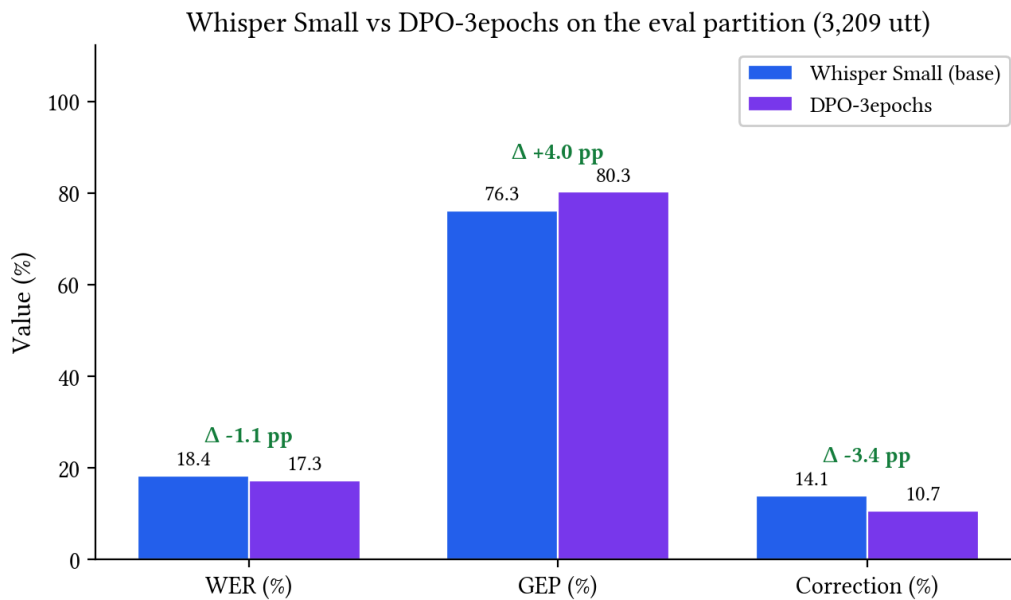


Figure 27: *Whisper Small (base) vs. the DPO-fine-tuned Whisper Small on the full eval partition. All three metrics move in the desirable direction simultaneously: WER drops by 1.1 pp, GEP gains 4.0 pp, and the absolute correction rate is reduced from 14.1% to 10.7%.*

Model	WER (%)	GEP (%)	Corr. (%)	Mut. (%)	EPR (%)
Whisper Small (base)	18.39	76.27	14.05	9.68	53.75
Whisper Small + DPO (3 ep)	17.27	80.30	10.70	9.00	57.53
Δ	-1.12	+4.03	-3.35	-0.68	+3.78

Every metric moves in the desirable direction. WER and the mutation rate both drop, confirming that the model has not become noisier; the auto-correction rate drops by a third in relative terms (14.1% $\rightarrow$ 10.7%); and the speech-phenomena preservation rate (EPR, the

fraction of disfluency / partial-word marks preserved, measured exactly as in the baseline chapter) rises by 3.8 pp as a side effect of the model becoming more faithful overall to what the speaker produced.

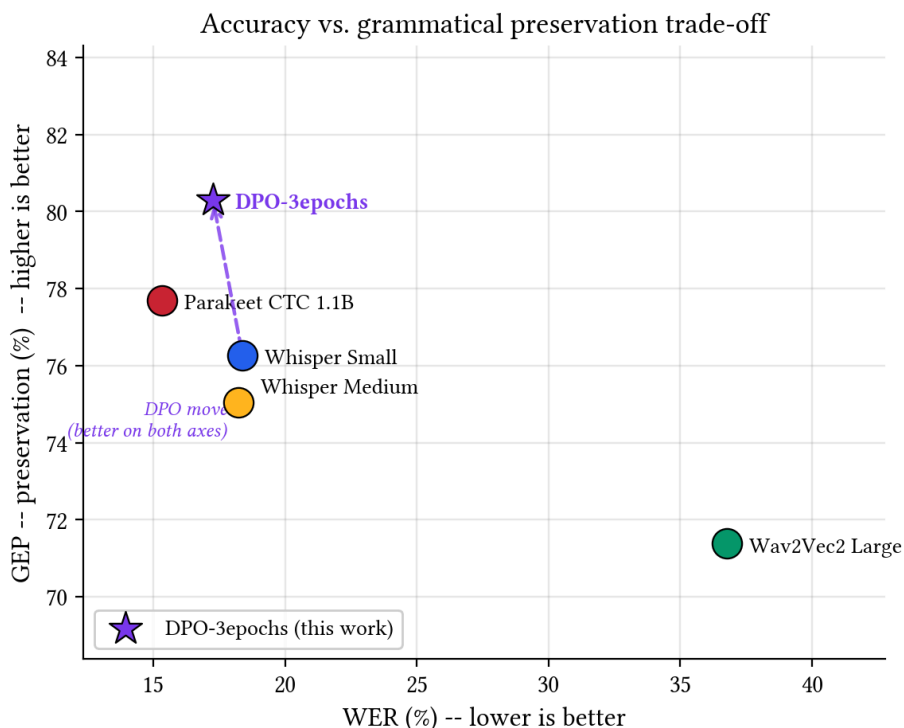


Figure 28: WER vs. GEP trade-off after adding the DPO model to the figure from the comparative evaluation chapter. The DPO point sits above and to the left of the Whisper Small baseline, dominating it on both axes; it also closes most of the gap to Parakeet CTC 1.1B on the GEP axis while approximately matching the same WER region.

The trade-off picture is the most informative summary. Figure 28 places the DPO model on the same WER-vs-GEP plot used in the baseline chapter (where Parakeet CTC 1.1B was identified as the Pareto-optimal baseline). The DPO model strictly dominates the original Whisper Small point: it is both higher on the GEP axis and to the left on the WER axis. It also pushes Whisper Small clearly above Parakeet CTC 1.1B on the GEP axis, at a WER cost that is within two percentage points of the Parakeet figure.

8.6. Where DPO Helps Most

Aggregate gains can hide unevenness across error types, so the per-category breakdown is informative.

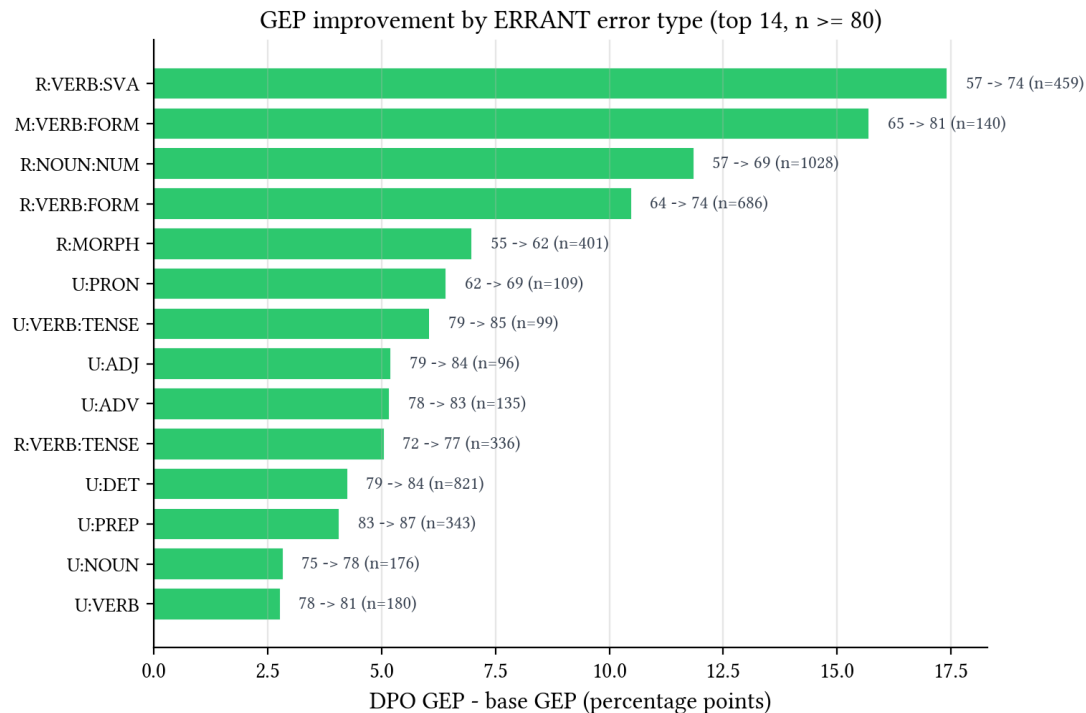


Figure 29: Per-category GEP improvement after DPO, restricted to ERRANT categories with at least 80 evaluated edits in the eval partition. Each row is annotated with the base GEP, the DPO GEP, and the number of evaluated edits.

Figure 29 shows the largest gains in the categories that motivated this work to begin with: morphosyntactic agreement errors (R:VERB:SVA +17 pp on 459 edits, R:NOUN:NUM +12 pp on 1,028 edits) and verb-form errors (R:VERB:FORM +10 pp on 686 edits, M:VERB:FORM +16 pp on 140 edits). These are exactly the categories where Whisper Small was most prone to silently restoring the standard form in the baseline evaluation, and they are also the categories that the LLM-restoration step is most reliable on (singular-plural and verb-form edits are short, well-defined token swaps).

The morphology bucket (R:MORPH) gains 7 pp; the “missing” and “unnecessary” buckets (U: and M: prefixes for function words such as determiners, prepositions and pronouns) move 4-6 pp each. No category in the figure has been made worse by DPO at the $n \geq 80$ threshold.

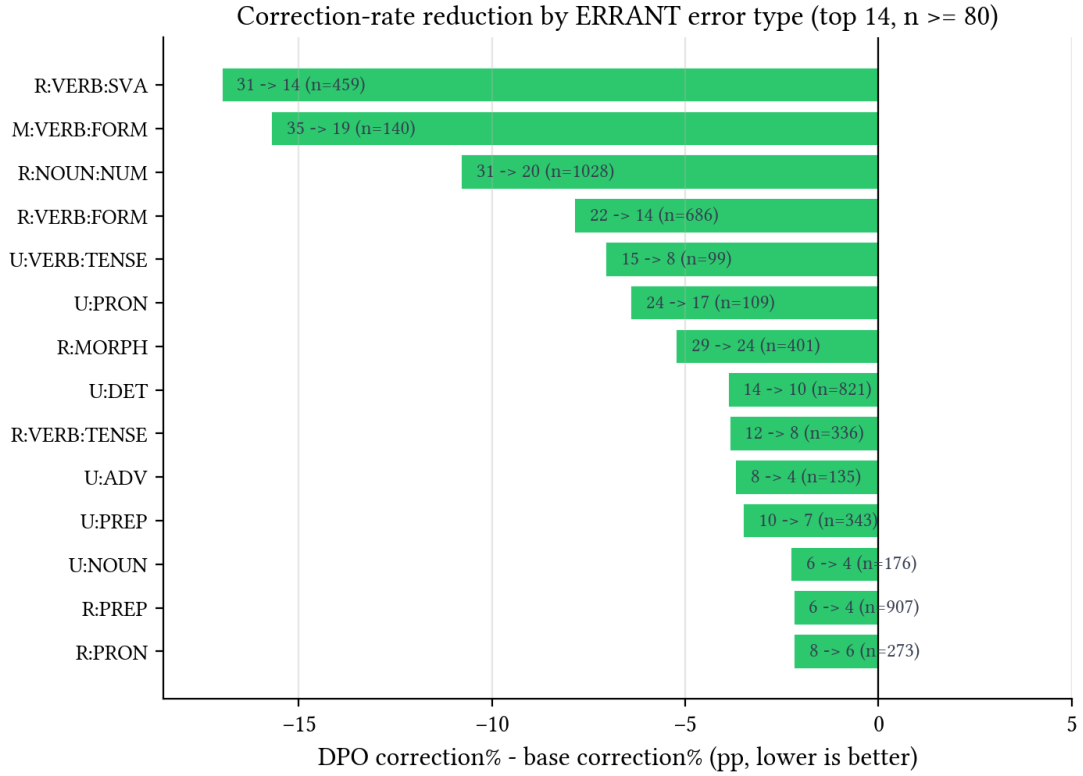


Figure 30: Reduction in the absolute auto-correction rate (DPO – base, in percentage points, lower is better) by ERRANT category. The reductions track the GEP gains in Figure 29 closely, which is the expected behavior: DPO does not push errors into mutated/spurious territory, it pushes them from corrected to preserved.

Figure 30 presents the same picture from the inverse angle: the absolute auto-correction rate drops by 17 pp on R:VERB:SVA (from 31% → 14%), 16 pp on M:VERB:FORM (35% → 19%), 11 pp on R:NOUN:NUM (31% → 20%), and 8 pp on R:VERB:FORM (22% → 14%). The mutation rate is essentially unchanged across the board, which means the model is not “preserving” errors by emitting noise; it is genuinely keeping the learner’s original word at positions where the baseline used to silently rewrite it.

8.7. Qualitative Examples

Qualitative examples: utterances where DPO restored the learner's error

SI114J-00762-P50019 (R:VERB:SVA)

reference (with error): a team leader is a person who work hard

base A team leader is a person who **works** hard, it should be kind, special, gorgeous and good working.

DPO A team leader is a person who **work** hard, it should be kind, special, gorgeous, good working.

SI127E-00368-P10008 (R:VERB:TENSE)

reference (with error): maybe i will like to to improve my writing skills and speaking skills

base Maybe I **would** like to improve my writing skills and speaking skills.

DPO Maybe I **will** like to improve my writing skills and speaking skills.

SI127E-00368-P10005 (R:VERB:FORM)

reference (with error): i'm interested in in learn about culture music and shows and everything

base I'm interested in **learning** about culture, music and shows and everything.

DPO I'm interested in **learn** about culture, music and shows and everything.

Figure 31: Three eval-partition utterances illustrating the change in behavior. In each case the learner produced an error that the baseline Whisper Small silently corrected; the DPO model preserves the original form. The rest of each transcription is essentially unchanged.

Figure 31 shows three utterances from the eval partition where DPO restored a learner error that the baseline corrected. In SI114J-00762-P50019 the learner says “a person who **work** hard”, a subject-verb agreement error; the baseline emits “a person who works hard”; DPO recovers “a person who work hard”. The rest of the transcription is identical between the two systems. The other two examples follow the same pattern with a verb-tense error (“**will** like” → baseline “would like”) and a verb-form error (“interested in **learn**” → baseline “interested in learning”). In all three cases the surrounding text, including punctuation, capitalization and trailing disfluency rendering, is unchanged: DPO has modified only the tokens that the training signal targeted.

8.8. Why the Recipe Works

A short comment on **why this recipe avoids the failure modes that less targeted approaches would run into**. A standard cross-entropy fine-tuning pass on the train partition references would push Whisper toward an output style it does not natively emit (lowercase, no punctuation, structural disfluency markers), and any per-token alignment scheme aimed at masking that style out would introduce silent bookkeeping bugs that con-

taminate the gradient. The DPO formulation avoids both problems at once: both sequences in every pair are in Whisper’s own output style by construction, so the loss never asks the model to change anything about how it writes English, only about which of two near-identical strings it prefers for a given audio.

A short comment on **why the WER also improves rather than degrading**. The naive expectation is that increasing GEP comes at a WER cost, because preserving a learner error contributes one substitution to the WER count. In practice DPO improves WER too, because (i) the corrected baseline was already wrong on the same edits (it produced “role” where the reference says “roles”, counting as one substitution either way), and (ii) the EPR figure shows that the policy is also more faithful to the non-grammatical content the learner produced (disfluencies, partials), which in some utterances was previously being smoothed away. The two effects compound.

A short comment on **the scope of the result**. The training set contains 1,274 preference pairs and the entire training run takes 40 minutes on a single 12 GB consumer GPU. This is not a result about scale; it is a result about pair construction. The same recipe, with a larger candidate pool, a larger LLM for the restoration step, or a Whisper-family model larger than Whisper Small, could plausibly close more of the remaining gap to Parakeet CTC 1.1B without changing the training procedure. The future work section in the next chapter returns to this question.

9. Discussion

9.1. Implications for Technology-Assisted Language Learning

The results, validated on 3,209 utterances and 10,397 GEP-scored grammatical errors (eval partition) with four baseline models and one targeted DPO *fine-tuning* experiment, have direct implications for the design of Computer-Assisted Language Learning (CALL) systems:

1. **ASR accuracy is not enough:** Whisper Small and Medium have nearly identical WER (18.39% vs. 18.23%) but qualitatively different behaviors regarding error preservation. Selecting an ASR model based solely on WER would be inadequate for pedagogical applications.
2. **Larger models are not always better within Seq2Seq:** Whisper Medium is **worse** than Whisper Small for this application. Its larger capacity (769M vs. 244M) allows it to correct 15% more learner errors (16.4% vs. 14.1%), reducing diagnostic information without improving transcription.
3. **Parakeet CTC 1.1B is the optimal model:** It combines the best WER (15.35%) with the highest grammatical preservation (77.7%) and a moderate correction rate (12.8%). It demonstrates that model scale and training data, rather than the Seq2Seq architecture, are the determining factors for accuracy in L2 speech.
4. **The CTC architecture is inherently more faithful:** Comparing models of similar scale (Parakeet CTC (1.1B) vs. Whisper Medium (769M)), Parakeet has a lower correction rate (12.8% vs. 16.4%) despite having more parameters. The difference stems from the absence of the autoregressive decoder, not the scale.
5. **Performance is maintained at reduced scale in CTC:** Parakeet TDT-CTC 110M (114M parameters) reproduces the performance of the 1.1B model (WER 15.67% vs. 15.35%, GEP 78.5% vs. 77.7%), demonstrating that the FastConformer CTC family scales efficiently and is viable for deployment on resource-constrained devices.
6. **Decoders penalize even at a small scale:** Canary 180M Flash, with a Transformer decoder of only 4 layers, shows higher auto-correction (13.4%) than Parakeet 110M (12.2%) despite having more parameters (182M vs. 114M). The decoder induces corrective bias regardless of size.
7. **Targeted DPO fine-tuning improves both axes simultaneously:** When the training signal is restricted to utterances where the baseline auto-corrects a whitelisted ERRANT

edit, and when supervision is expressed as a preference between two near-identical Whisper-style strings rather than as an alignment-dependent cross-entropy loss, a small fine-tuning pass (1,274 pairs, 40 minutes on a single 12 GB GPU) raises Whisper Small’s GEP from 76.3% to 80.3% while **also** lowering its WER from 18.39% to 17.27%. Architectural selection remains the largest lever, but a focused-scope DPO pass on top of an autoregressive baseline closes a meaningful fraction of the gap to the best CTC model.

8. **Agreement and morphology errors are the most vulnerable:** R:NOUN:NUM (30–35%), R:VERB:SVA (22–34%), and R:MORPH (23–32%) are **local** errors solvable with immediate context, which can be captured by both autoregressive decoders and well-trained CTC models.

9.2. Limitations

- **Omission classification limits:** The alignment heuristic for missing-word errors (M : *xxx*) relies on a local search window between successfully transcribed neighboring words, which can misclassify insertions if the surrounding audio is highly distorted or deleted by the model.
- **Asymmetry of GEC annotations:** The dev partition contains only 64 GEP-scored grammatical errors compared to 10,397 in eval, preventing robust cross-validation of the GEP metric.
- **Absence of CTC models with external LM:** Wav2Vec2 with an external language model was not included, which could offer an interesting middle ground.
- **English only:** Results are specific to learners of English and models trained on English.
- **Hardware constraints:** The RTX 3060 (12 GB) GPU restricted the exploration of larger Whisper variants (Whisper Medium / Large-V3) and of higher-rank LoRA configurations for the DPO recipe.

9.3. Future Work

1. **Scaling the DPO recipe:** The 1,274-pair training set was deliberately limited by the size of the S&I train partition. Building a larger candidate pool (additional L2 corpora, or synthetic L2 utterances generated with TTS over LLM-injected grammatical errors) is the most direct way to test whether the gains observed here continue to scale. A larger pool would also let the LLM-restoration step be more selective, dropping pairs with ambiguous edits.
2. **Applying the recipe to larger Whisper variants:** The same DPO recipe should be tested on Whisper Medium / Large-V3, which auto-correct more aggressively than

Whisper Small in the baseline evaluation, to determine whether the gap to Parakeet can be closed entirely.

3. **Post-processing as a complementary path:** For deployments where the ASR model cannot be retrained, a post-processing module that detects and reverts grammatical corrections by comparing the ASR output against an acoustic or phonetic re-scoring of the audio could complement the DPO recipe rather than replace it.
4. **Constrained decoding:** Modifying the decoding strategy (*constrained beam search*) to favor transcriptions that preserve erroneous forms when the acoustic evidence supports them, in concert with a DPO-fine-tuned policy.
5. **Granular analysis by proficiency level:** The analysis presented in this work uses the overall SLA score; a future study could leverage section-specific scores to determine if the type of oral task influences error preservation, or correlate with external CEFR levels, and confirm that the DPO recipe's gains are uniform across proficiency bands.
6. **Cross-lingual Embedding Interference:** Furthermore, an interesting avenue for future work concerns the cross-lingual interference in embedding spaces. If an ASR model has been trained exclusively on English and it is fed English pronunciations from Spanish students, the model has no choice but to map the acoustic input into the English language embedding space. However, if a model has been trained multilingually (e.g., on English, Spanish, and optionally other languages), the English spoken by the learners will generally be heavily influenced by Spanish phonetics. Consequently, the acoustic embeddings circulating in the model might be closer to the Spanish space than the English one. This cross-lingual interference could end up negatively affecting the model's performance on L2 English. Investigating how multilingual models navigate these overlapping phonetic spaces when processing accented speech could yield significant improvements for CALL systems.

10. Conclusions

This work presents an evaluation *pipeline* that combines two complementary metrics: WER and GEP. These are used to analyze the behavior of ASR models when handling language learner speech. The GEP metric, based on ERRANT, enables granular quantification of which types of grammatical errors are auto-corrected by each model.

The results, validated on 3,209 utterances and 10,397 GEP-scored grammatical errors (eval partition) from the Speak & Improve corpus with four models, confirm the hypothesis that Seq2Seq models with autoregressive decoders (Whisper) auto-correct grammatical errors more frequently than CTC models. Whisper Medium (16.4%) corrects more than Whisper Small (14.1%), which in turn corrects more than Parakeet CTC (12.8%), which corrects more than Wav2Vec2 (2.9%). The hierarchy is clear: more linguistic capacity in the decoder implies more auto-correction.

The most significant finding is that **Parakeet CTC 1.1B dominates the Pareto frontier**: it achieves the best WER (15.35%) among all evaluated models **and** the highest grammatical preservation (77.7%), with a lower auto-correction rate than both Whisper models. This demonstrates that the CTC architecture, given sufficient training data (64,000 hours), offers the best trade-off for language tutoring applications.

Granular analysis by ERRANT error type (39 scored types) identifies agreement errors (R:NOUN:NUM, R:VERB:SVA), morphology (R:MORPH), and verb form (R:VERB:FORM) as the most vulnerable to auto-correction, with rates of 20–35% depending on the model. Lexical selection errors (R:PREP, R:VERB, R:NOUN) are preserved in >75% of cases.

The analysis by speaker proficiency level, based on continuous SLA scores, reveals that **errors from lower-level speakers are preserved worse across all models**: GEP in low-proficiency speakers ranges between 65.2% (Wav2Vec2) and 76.2% (Parakeet), compared to 75.5–78.6% in high-proficiency speakers. Parakeet CTC 1.1B shows the lowest variation across levels ($\Delta = 2.2$ pp), reinforcing its suitability for tutoring applications where robustness to learner level is critical.

The evaluation of lightweight models reveals that **Parakeet TDT-CTC 110M reproduces the performance of the 1.1B model** with only 10.7% of its parameters: WER 15.67% vs. 15.35%, GEP 78.5% vs. 77.7%, and correction rate 12.2% vs. 12.8%. This result validates the deployment of compact CTC models in resource-constrained environments without sacrificing error preservation. In contrast, Canary 180M Flash (182M, encoder-decoder) shows

a significantly higher WER (25.47%) and comparable auto-correction (13.4%), confirming that even minimal decoders induce corrective behavior.

A targeted Direct Preference Optimization (DPO) *fine-tuning* pass on Whisper Small, trained on 1,274 selectively constructed preference pairs in roughly 40 minutes on a single 12 GB consumer GPU, **improves both metrics simultaneously**: WER drops from 18.39% to 17.27% and GEP rises from 76.3% to 80.3%. The absolute auto-correction rate drops by a third in relative terms (14.1% $\rightarrow$ 10.7%), with the largest gains concentrated in exactly the morphosyntactic categories that the baseline was most prone to over-correcting (R:VERB:SVA -17 pp, M:VERB:FORM -16 pp, R:NOUN:NUM -11 pp). Two design choices are responsible for this outcome: restricting the training data to only those utterances where the baseline auto-corrects a whitelisted ERRANT edit, and casting supervision as a preference between two Whisper-style strings that differ at exactly the error site, sidestepping the alignment problems that would dominate any cross-entropy approach on these references. The DPO recipe does not contradict the central architectural finding; it complements it. On the WER-GEP plot, the DPO-fine-tuned Whisper Small sits above and to the left of the original baseline, but it also moves above Parakeet CTC 1.1B on the GEP axis at a WER cost of less than two percentage points, narrowing the gap to the Pareto-optimal baseline. For deployments where committing to a Whisper-class model is non-negotiable (e.g. punctuated and cased output, or established Whisper infrastructure), a focused-scope DPO pass closes most of the remaining preservation gap to the best CTC baseline.

These findings together reinforce the central conclusion: for language tutoring applications, **architectural selection** (specifically large-scale CTC models like Parakeet) is the single largest lever, but a targeted DPO pass with carefully constructed preference pairs is a viable second lever that can recover most of the remaining gap on a Whisper-class model. The developed pipeline, integrating WER and GEP metrics, provides the necessary tools for this evaluation and remains available for future research involving larger corpora, larger base models, or alternative strategies like post-processing and constrained decoding.

11. References

1. Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006). *Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks*. In Proceedings of the 23rd International Conference on Machine Learning (ICML), pp. 369–376. <https://doi.org/10.1145/1143844.1143891>
2. Morris, A. C., Maier, V., & Green, P. (2004). *From WER and RIL to MER and WIL: Improved Evaluation Measures for Connected Speech Recognition*. INTERSPEECH.
3. Xiong, W., Droppo, J., Huang, X., Seide, F., Seltzer, M., Stolcke, A., Yu, D., & Zweig, G. (2017). *The Microsoft 2016 Conversational Speech Recognition System*. In Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 5255–5259. <https://doi.org/10.1109/ICASSP.2017.7953159>
4. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). *Robust Speech Recognition via Large-Scale Weak Supervision*. In Proceedings of the 40th International Conference on Machine Learning (ICML 2023), PMLR 202: 28492–28518.
5. Baevski, A., Zhou, Y., Mohamed, A., & Auli, M. (2020). *wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations*. NeurIPS.
6. Rekesh, D., Koluguri, N. R., Krizan, S., Majumdar, S., Noroozi, V., Huang, H., Hrinchuk, O., Puvvada, K. C., Kumar, A., Balam, J., & Ginsburg, B. (2023). *Fast Conformer with Linearly Scalable Attention for Efficient Speech Recognition*. In 2023 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU), pp. 1–8. <https://doi.org/10.1109/ASRU57964.2023.10389701>
7. Xu, H., Jia, F., Majumdar, S., Huang, H., Watanabe, S., & Ginsburg, B. (2023). *Efficient Sequence Transduction by Jointly Predicting Tokens and Durations*. In Proceedings of the 40th International Conference on Machine Learning (ICML), pp. 38462–38484.
8. Puvvada, K. C., Żelasko, P., Huang, H., Hrinchuk, O., Koluguri, N. R., Dhawan, K., Majumdar, S., Rastorgueva, E., Chen, Z., Lavrukhin, V., Balam, J., & Ginsburg, B. (2024). *Less is More: Accurate Speech Recognition & Translation Without Web-Scale Data*. arXiv preprint arXiv:2406.19674.
9. Jeffries, N., King, E., Kudlur, M., Nicholson, G., Wang, J., & Warden, P. (2024). *Moonshine: Speech Recognition for Live Transcription and Voice Commands*. arXiv preprint arXiv:2410.15608.
10. Eskenazi, M. (2009). *An Overview of Spoken Language Technology for Education*. Speech Communication, 51(10), 832–844. <https://doi.org/10.1016/j.specom.2009.04.005>

11. Qian, M., Knill, K., Banno, S., Tang, S., Karanasou, P., Gales, M. J. F., & Nicholls, D. (2024). *Speak & Improve Challenge 2025: Spoken Language Assessment and Feedback*. arXiv preprint arXiv:2412.11985.
12. Knill, K., Nicholls, D., Gales, M. J. F., Qian, M., & Stroinski, P. (2024). *Speak & Improve Corpus 2025: an L2 English Speech Corpus for Language Assessment and Feedback*. arXiv preprint arXiv:2412.11986.
13. Bryant, C., Felice, M., & Briscoe, T. (2017). *Automatic Annotation and Evaluation of Error Types for Grammatical Error Correction*. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 793–805, Vancouver, Canada.
14. Hu, E. J., et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR.
15. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS.
16. Raschka, S. (2023). *Practical Tips for Finetuning LLMs Using LoRA (Low-Rank Adaptation)*. Lightning AI / Ahead of AI.
17. Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS.
18. McFee, B., Raffel, C., Liang, D., Ellis, D. P. W., McVicar, M., Battenberg, E., & Nieto, O. (2015). *librosa: Audio and Music Signal Analysis in Python*. In Proceedings of the 14th Python in Science Conference, pp. 18–24. <https://doi.org/10.25080/Majora-7b98e3ed-003>
19. jitsi. *jiwer: Evaluate Your Speech-to-Text System with Similarity Measures Such as Word Error Rate (WER)* [Computer software]. GitHub. <https://github.com/jitsi/jiwer>